MODULE 06 · AMBITIOUS ENTREPRENEURSHIP

Prototyping with AI

What is the fastest thing you can build to learn?

COURSE PATH

00 01 02 03 04 05 **06** 07 08 09 10 11 12

COURSE MATERIAL · SCAN NOW

All slides live on GitHub

The newest version, free, no account needed

github.com/BaNburger/
AmbitiousEntrepreneurship

1. **Open the link**
   github.com/BaNburger/AmbitiousEntrepreneurship
2. **Go to modules**
   then 06-prototyping
3. **Click the lecture PDF**
   M06_Lecture_Prototyping.pdf, download top right

Public course repository of Ambitious Entrepreneurship. The QR code opens github.com/BaNburger/AmbitiousEntrepreneurship.

WHERE WE ARE · THE COURSE

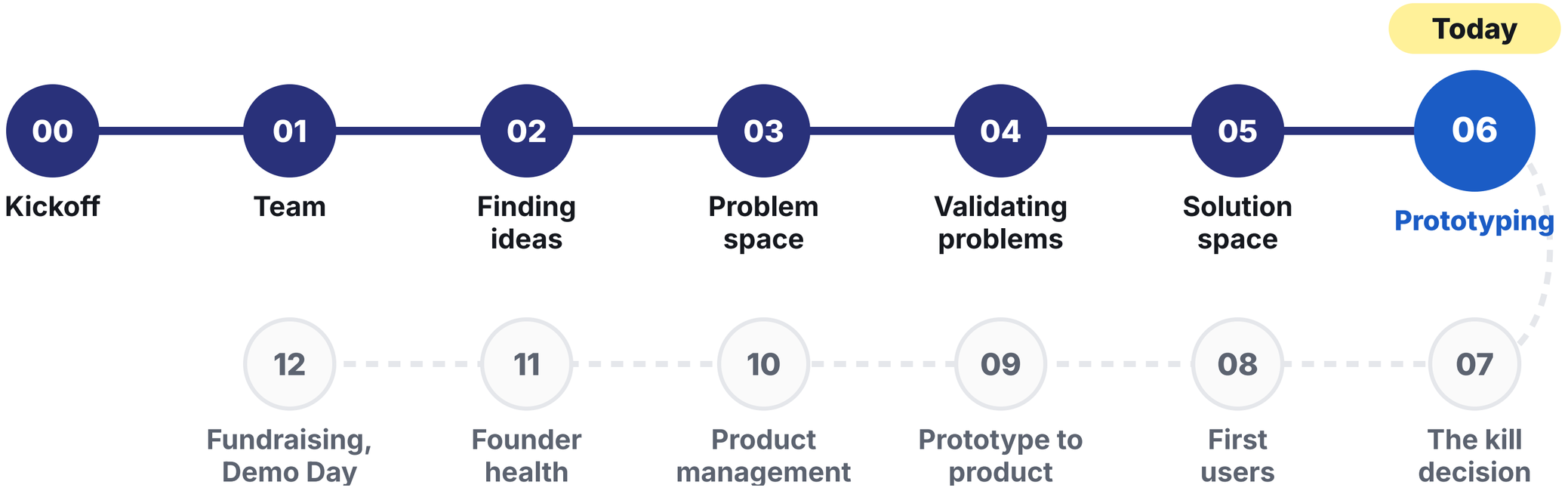
You are here: the dummy has to work

Thirteen sessions from first idea to first revenue. What teams have by now:

A problem people confirmed
Problem space (M03), Validating problems (M04)

Sign-ups on a landing page
Validating problems (M04)

A winning click dummy
Solution space (M05)



CHECK-IN · SHOW OF HANDS

Where did your last test land?

Your line is the result you wrote down before the test. Below it, you change course

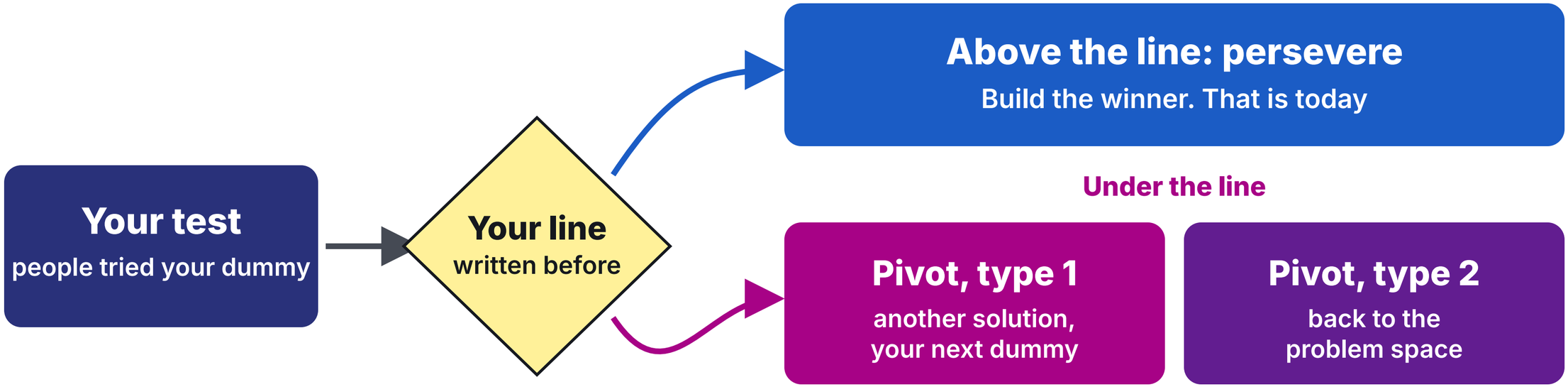
1. Above my line · persevere, build it today
2. Under my line · pivot, and I learned it cheaply
3. No line written · write one before today's first user

2:00

PIVOT OR PERSEVERE

Under your line? That is a decision, not a failure

Every test ends in one of two moves, and both are progress



Both are progress. The only miss: no line, so no decision

FRAMEWORK Pivot or persevere after Ries, E. (2011), The Lean Startup. Pivot types 1 and 2 as in the course Glossary and Solution space (M05).

ACCOUNTABILITY

Report what you did, not what you planned



User interactions?

What did you learn?



New revenue?

Why did they pay?



Shipped?

Why does it matter?

🦄 **Report target vs. actual in small groups**

CHAPTER 1 OF 9

Previously on Ambitious Entrepreneurship

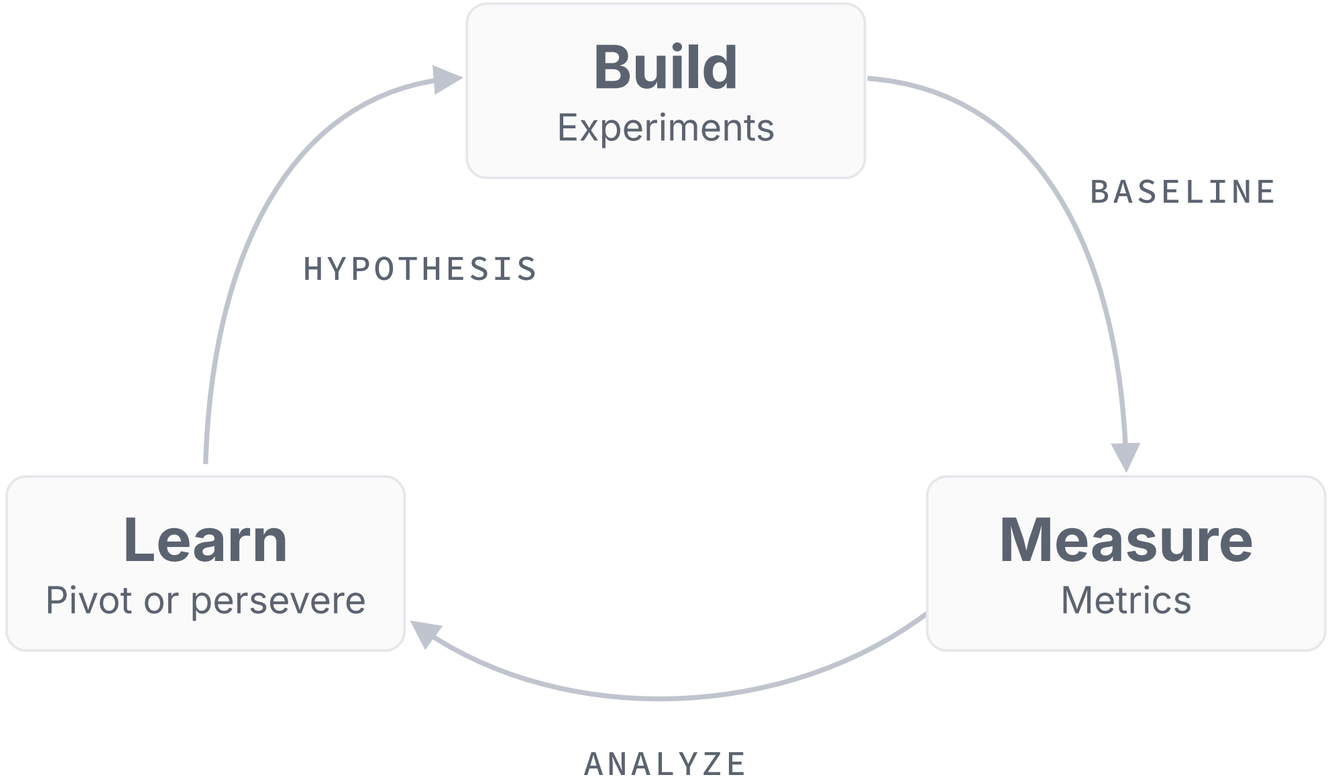
Four pictures from earlier sessions that you need to follow today.

01 **Previously on Ambitious Entrepreneurship**
02 Your presentations
03 **Map it: the parts of every app**
04 **Shrink it: the cheapest version**
05 **Build it: you steer the agent**
06 **Ship it: into users' hands**
07 Wrap-up and check-out
08 Workshop: build it, then hand it over
09 Case discussion

PREVIOUSLY · KICKOFF (M00)

Every idea is a bet you test in loops

Build the smallest test, measure what people do, decide

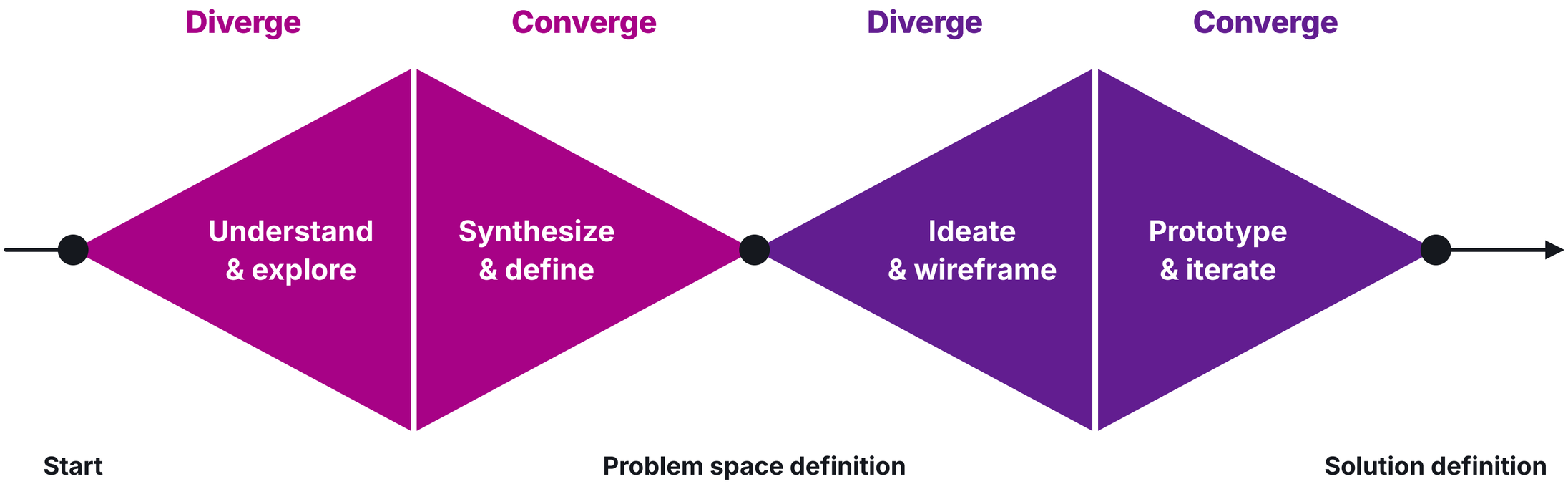


FRAMEWORK Ries, E. (2011). The Lean Startup. Crown Business. Course graphic of Kickoff (M00).

PREVIOUSLY · PROBLEM SPACE (M03) AND SOLUTION SPACE (M05)

First the problem, then the solution

Teams confirmed a problem people have, then tried several ways to solve it

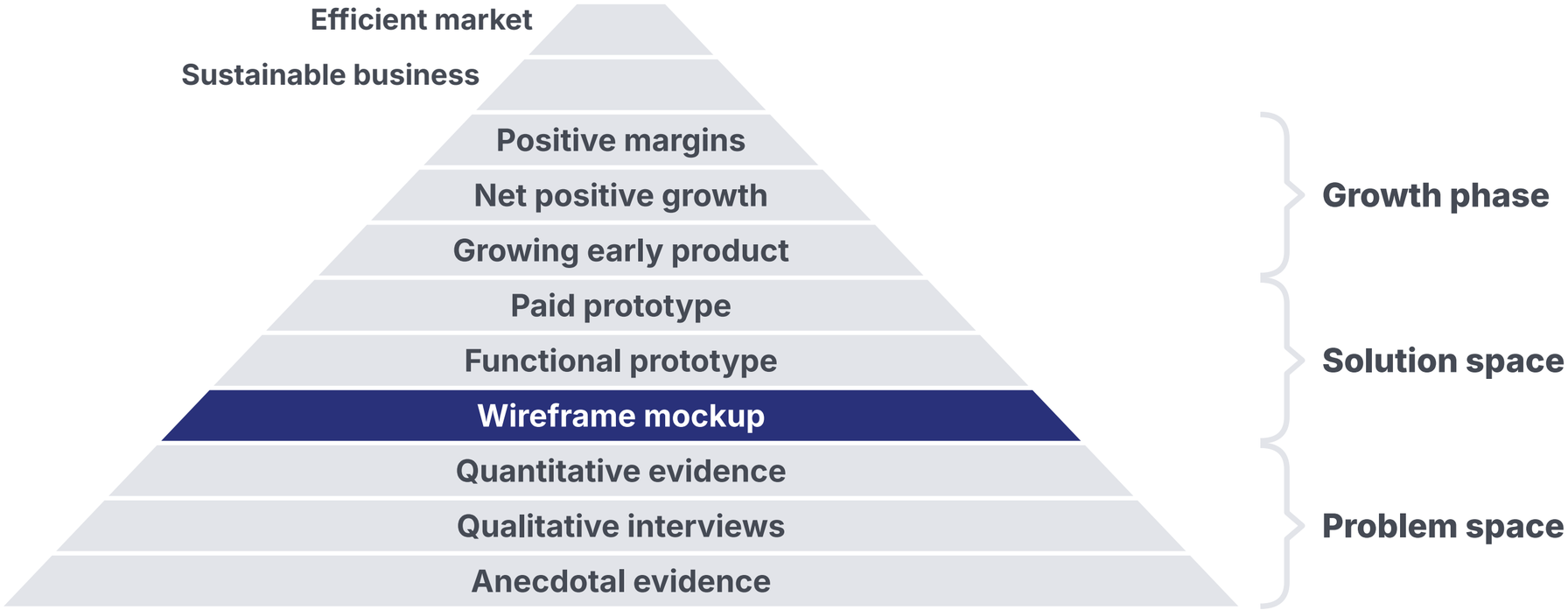


Double diamond after Design Council (2004), course graphic of Finding ideas (M02) and Solution space (M05).

PREVIOUSLY · VALIDATING PROBLEMS (M04) AND SOLUTION SPACE (M05)

From the mock-up to a slice that works

Higher rungs count what people did, not what they said



PREVIOUSLY · SOLUTION SPACE (M05)

Now we are building to get users

A click dummy looks real but does nothing. Teams tried several, kept one, and asked people to commit

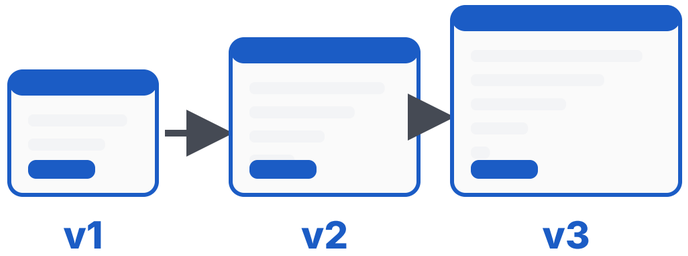
Build to find a seed
many dummies, most thrown away

Build to improve
one winner, better each round

Build to get users
one slice that works



Solution space (M05)



Solution space (M05)



Prototyping (M06), today

people use it alone, and come back

PREVIOUSLY · SOLUTION SPACE (M05)

You cut it to one feature. Build exactly that one

The MVP is the smallest thing someone could pay for. Today it has to work

8 features you could build

- ~~Login~~
- ~~Dashboard~~
- ~~AI scoring~~
- ~~Settings~~
- ~~Team invites~~
- ~~Dark mode~~
- ~~Export~~
- ~~Calendar sync~~

1 you build today, and it works

THE ONE FEATURE

Alert when a finalist goes quiet

- 10 times better at one thing
- The one users care about
- Ugly and slow is fine
- Tech doesn't matter

PREVIOUSLY · KICKOFF (M00)

~~But I'm not an engineer...~~

Yes, you can build it!

~~But we don't have a GmbH...~~

Yes, you can ship it!

~~But we don't have a patent...~~

Customers don't buy patents!

~~But we didn't get a grant...~~

Customers can pay you!

TODAY'S QUESTION · HANDS

What is the fastest thing you can build to learn?

1:00

CHAPTER 2 OF 9

Your presentations

Your Solution space (M05) line next to the asks people accepted, three minutes per team.

01 ~~Previously on Ambitious Entrepreneurship~~
02 **Your presentations**
03 **Map it: the parts of every app**
04 **Shrink it: the cheapest version**
05 **Build it: you steer the agent**
06 **Ship it: into users' hands**
07 Wrap-up and check-out
08 Workshop: build it, then hand it over
09 Case discussion

YOUR PRESENTATIONS · MAKE IT CREDIBLE

Show us where you're at in 3 minutes or less!

3:00

Facts
What the segment uses today and what it costs, with its source

12 Numbers
Guided demos per student, demos per dummy, asks accepted per dummy next to the line

Quotes
Two things people said while clicking, with roles, not names

Images
The winning dummy's key screen and the ask screen

Verdict: which dummy passed, which dropped (type 1), or back to the problem (type 2)

What did you do?

What did you learn? Where did the AI second opinion disagree?

How can we help?

INPUT 1 OF 4 · CHAPTER 3 OF 9

Map it: every application ever

Know the parts of every app, and which must really work.

01 ~~Previously on Ambitious Entrepreneurship~~
02 ~~Your presentations~~
03 **Map it: the parts of every app**
04 **Shrink it: the cheapest version**
05 **Build it: you steer the agent**
06 **Ship it: into users' hands**
07 Wrap-up and check-out
08 Workshop: build it, then hand it over
09 Case discussion

INPUT 1 OF 4 · MAP IT · THREE QUESTIONS

Before you build: what is an app made of?

1

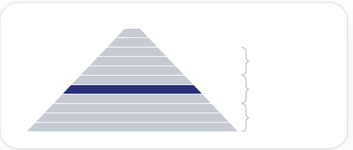
What is every app made of?

2

Which part must really work in a prototype?

3

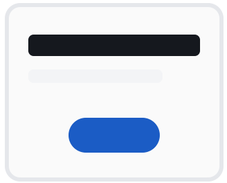
What must never break?



OVERVIEW · PROTOTYPES

Every prototype answers one question

Each climbs a rung. A lab demo shows technology, not a user's outcome

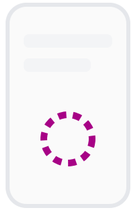


Landing page

rung 3

Do they want it?

Validating problems (M04)



Click dummy

rung 4

Do they get it?

Solution space (M05)

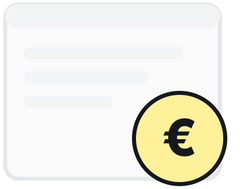


Functional prototype

rung 5

Do they use it alone, and come back?

Prototyping (M06), today



Paid prototype

rung 6

Will they pay?

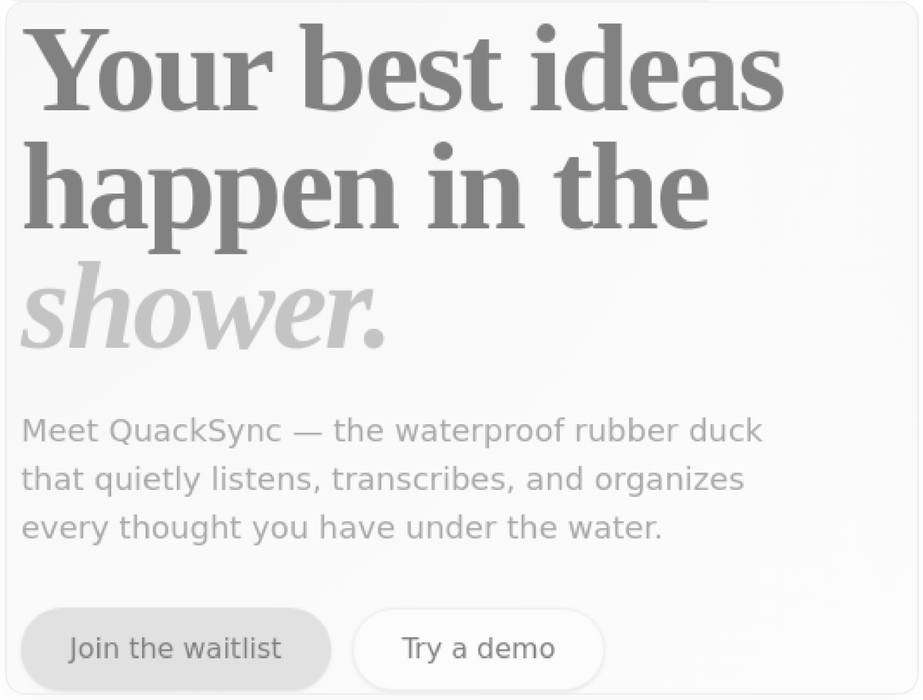
First users (M08)

MICRO CASE · QUACKSYNC, A MADE-UP VENTURE

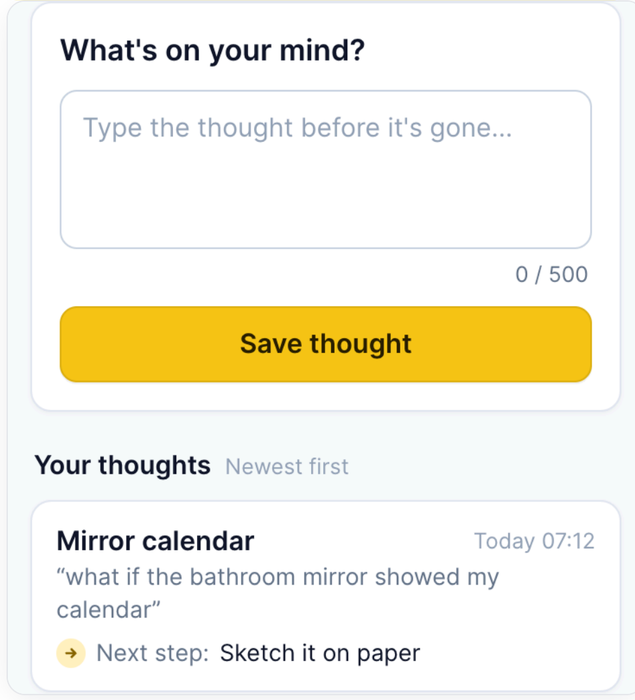
QuackSync: from a page to a feature that works

Problem: your best ideas come in the shower and are gone when you are dressed

BEFORE · SOLUTION SPACE (M05)



TODAY · PROTOTYPING (M06)

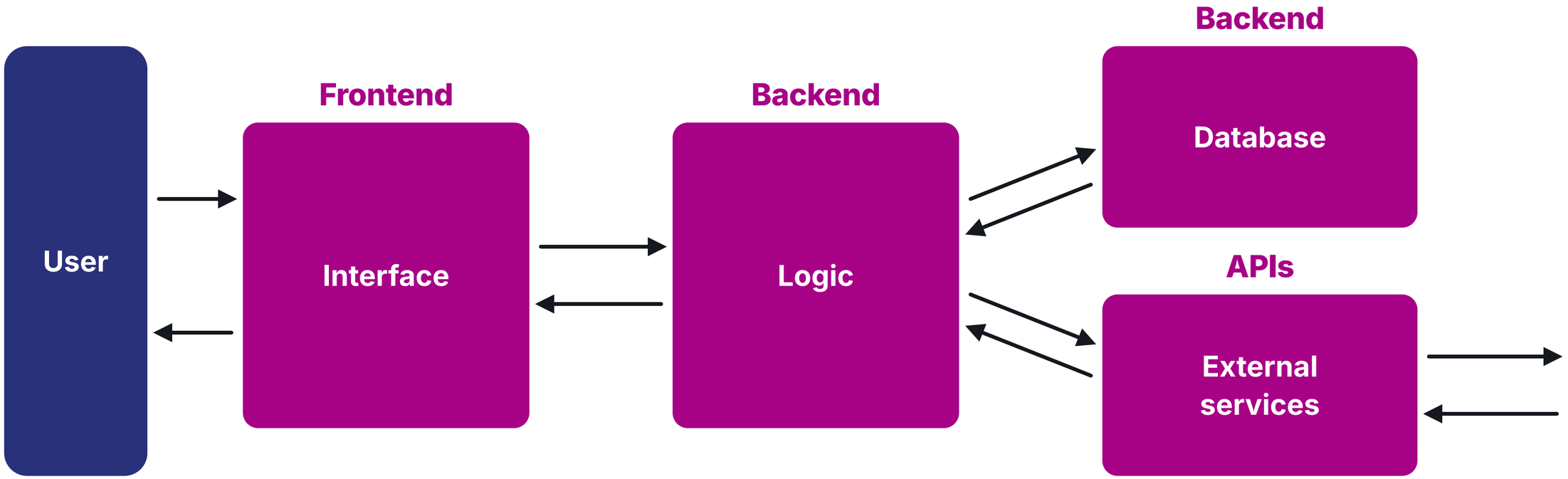


ILLUSTRATIVE QuackSync is made up. Left: built with Lovable, 23 June 2026, as shown in Solution space (M05), cropped. Right: screen prototyped for this deck.

YOUR SHORTCUT TO THINKING LIKE AN ARCHITECT

Every application has the same five parts

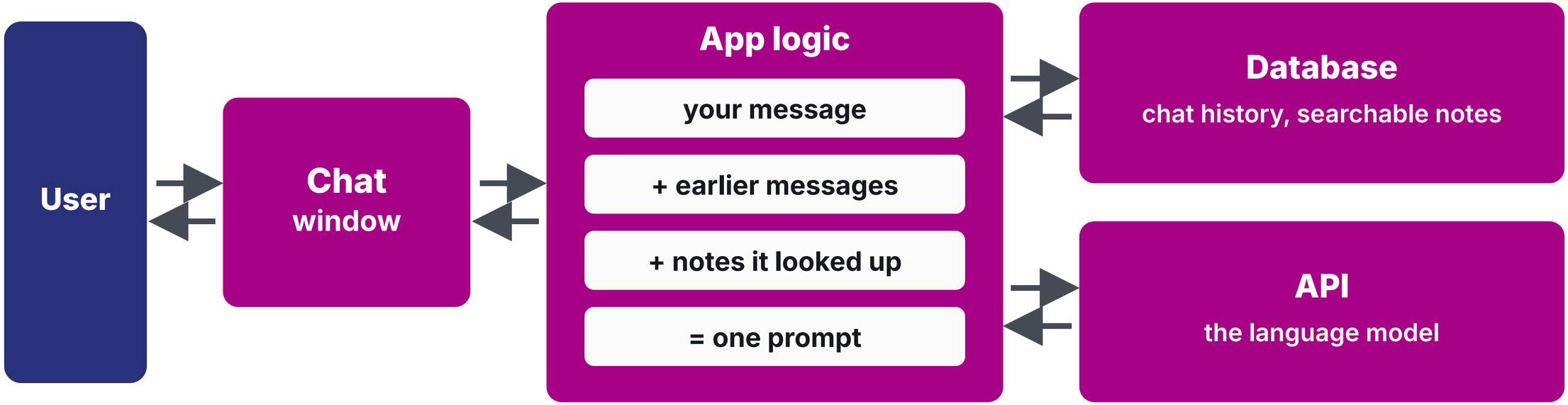
User, interface, backend, database, external services. Draw yours first

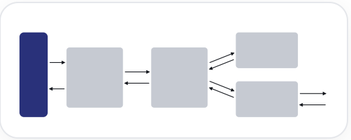


WORKED EXAMPLE

Even an AI chatbot is the same five parts

The magic is one prompt, built by the logic and sent to a model





EVERY APPLICATION EVER · 1 USER · IN A RESTAURANT

Every restaurant starts with a hungry guest

Not with the kitchen



IN A RESTAURANT

A guest who wants a good meal, fast, and knows roughly what

IN YOUR APP

The user brings the input and wants an outcome back

IN QUACKSYNC

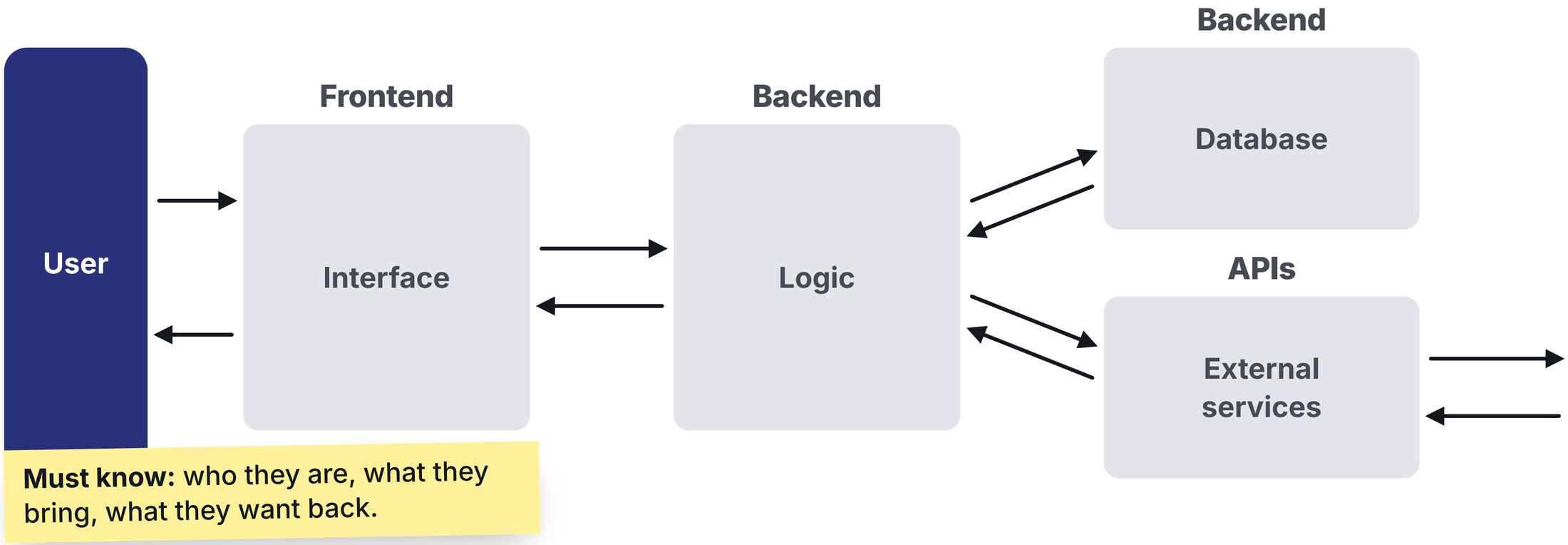
Someone in the shower with a thought they don't want to lose

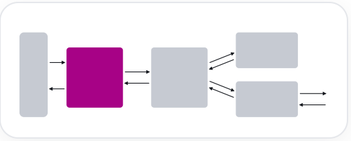
ILLUSTRATIVE A restaurant as the analogue of Every application ever, course graphic. Photo: StockSnap on Pixabay.

EVERY APPLICATION EVER · 1 USER

User: brings the input, wants an outcome

Not built, but the part you understand first





EVERY APPLICATION EVER · 2 INTERFACE · IN A RESTAURANT

The menu and the counter take the order

Where the order goes in and the meal comes out



IN A RESTAURANT

The menu shows what you can get, the counter takes the order and hands out the meal

IN YOUR APP

The interface asks for the input and shows the outcome

IN QUACKSYNC

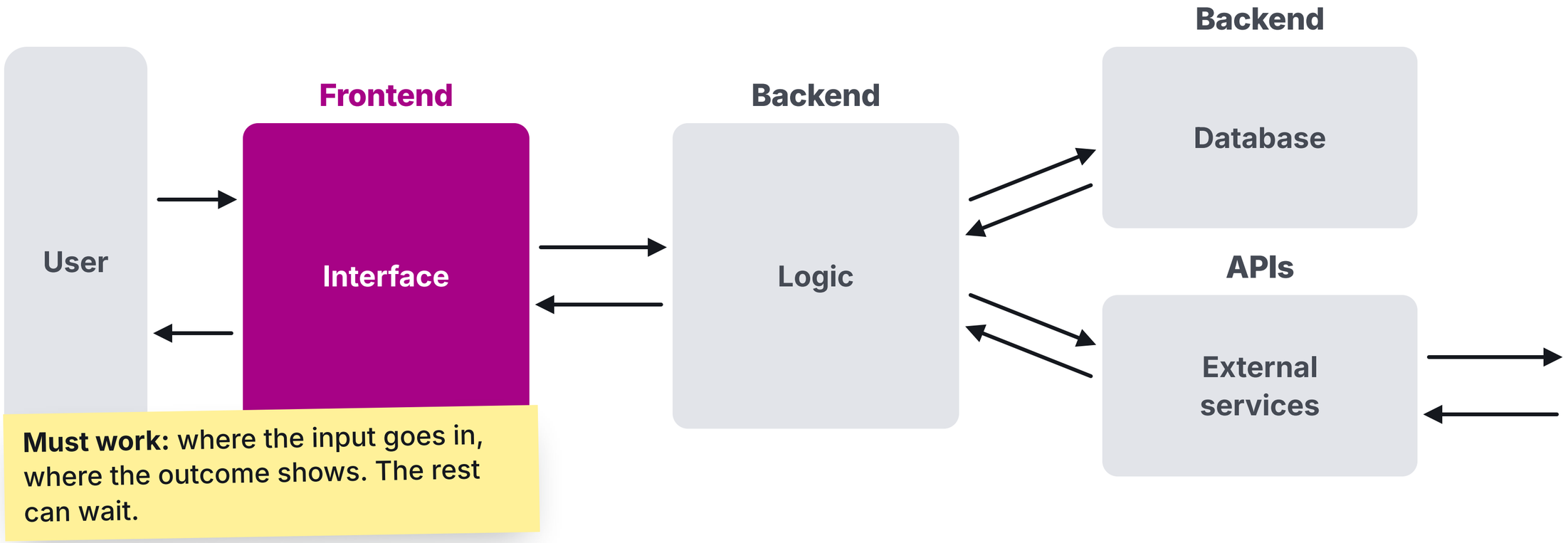
One box for the thought, one list with titles and next steps

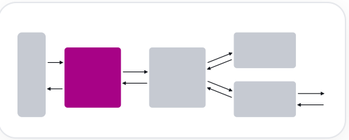
ILLUSTRATIVE A restaurant as the analogue of Every application ever, course graphic. Photo: StockSnap on Pixabay.

EVERY APPLICATION EVER · 2 INTERFACE

Interface: it asks for the input

Also called the frontend. Everything the user sees and touches

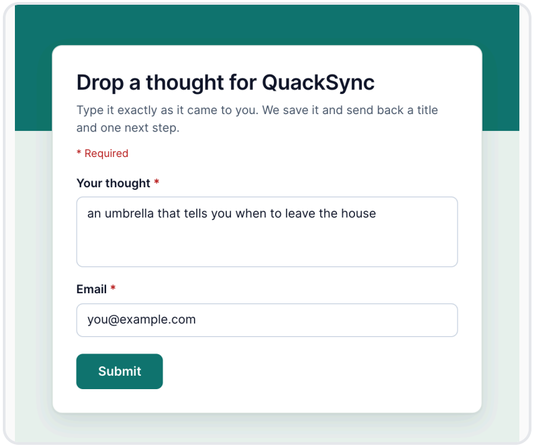


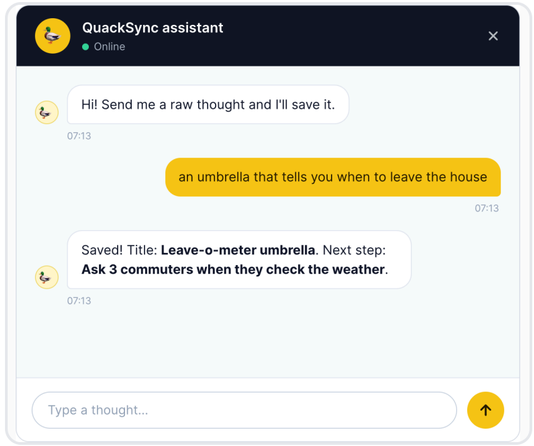


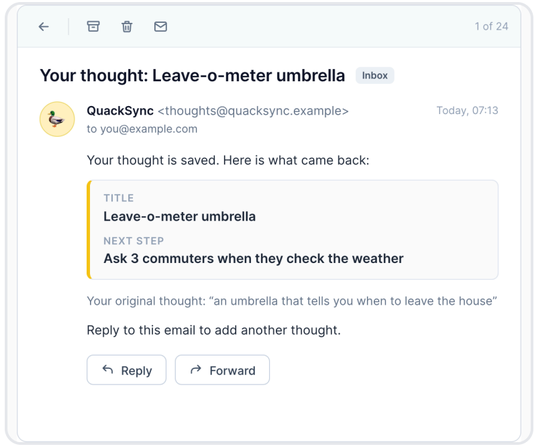
INTERFACE · IT NEED NOT BE A WEBSITE

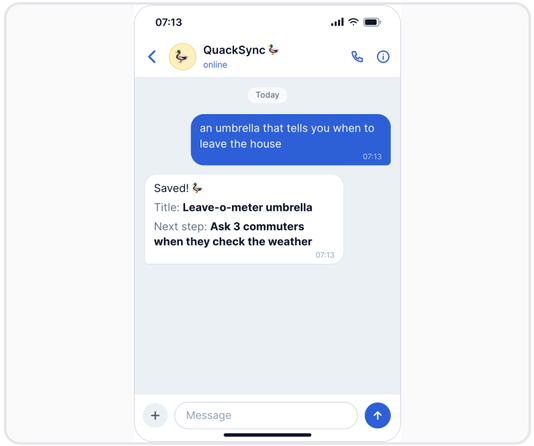
The interface can be a form, a chat or an email

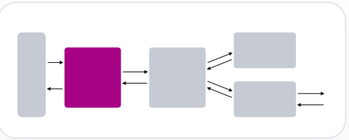
Pick the one your users already open every day











INTERFACE · QUACKSYNC

One box in, one list out, nothing else

No login, no settings, no dark mode

What's on your mind?

an umbrella that tells you when to leave the house

50 / 500

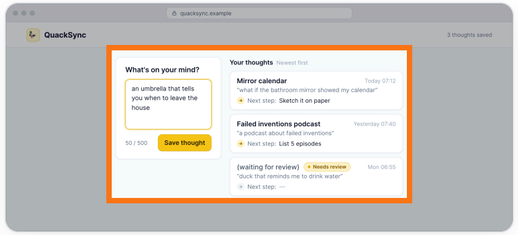
Save thought

Your thoughts Newest first

Mirror calendar Today 07:12
“what if the bathroom mirror showed my calendar”
Next step: Sketch it on paper

Failed inventions podcast Yesterday 07:40
“a podcast about failed inventions”
Next step: List 5 episodes

(waiting for review) Needs review Mon 06:55
“duck that reminds me to drink water”
Next step: —

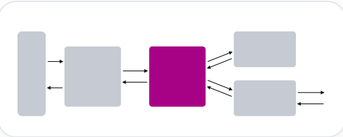


Input: one box, 500 characters

Outcome: a title and one next step

By hand: the yellow "waiting for review"

ILLUSTRATIVE QuackSync is made up. Screen prototyped for this deck.



EVERY APPLICATION EVER · 3 BACKEND · IN A RESTAURANT

Run the kitchen by hand before you build it

Then automate only the part that hurts

IN A RESTAURANT

The kitchen turns an order into a meal. McDonald's first laid it out in chalk, by hand

IN YOUR APP

The backend turns the input into the outcome

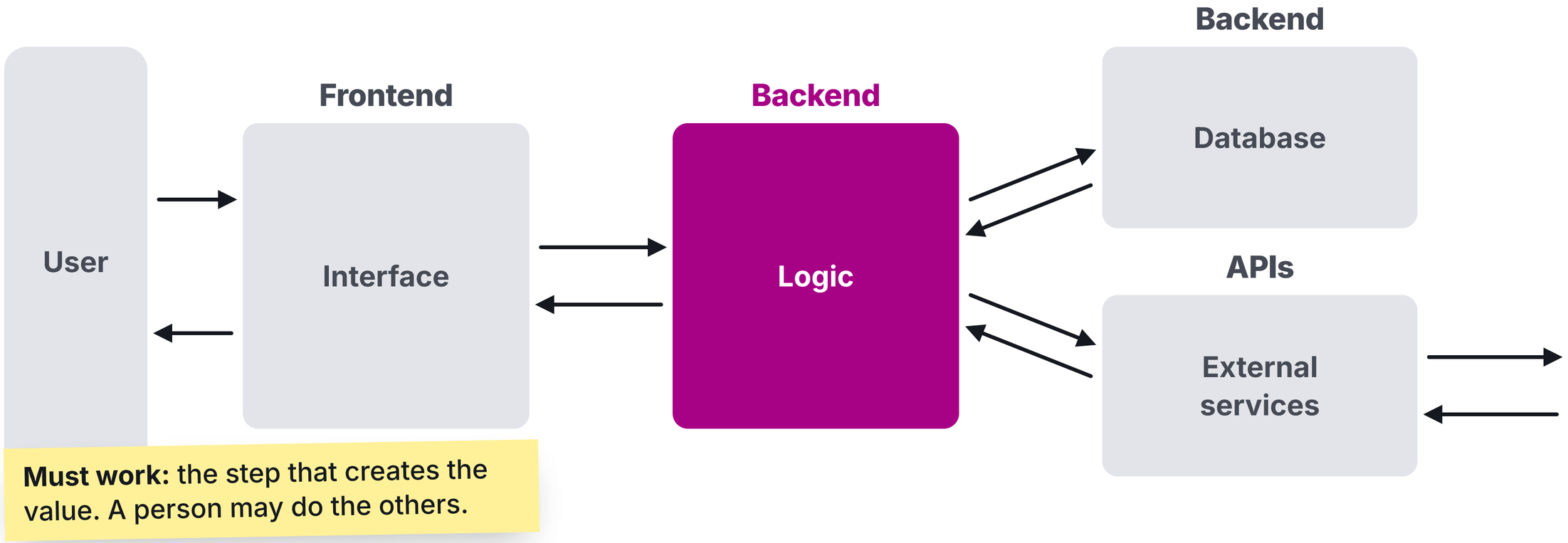
IN QUACKSYNC

Check the thought, ask the model, save it

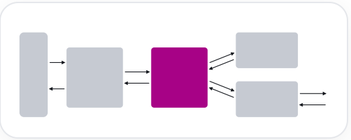
EVERY APPLICATION EVER · 3 BACKEND

Backend: it turns input into value

Also called the logic. The steps from input to outcome



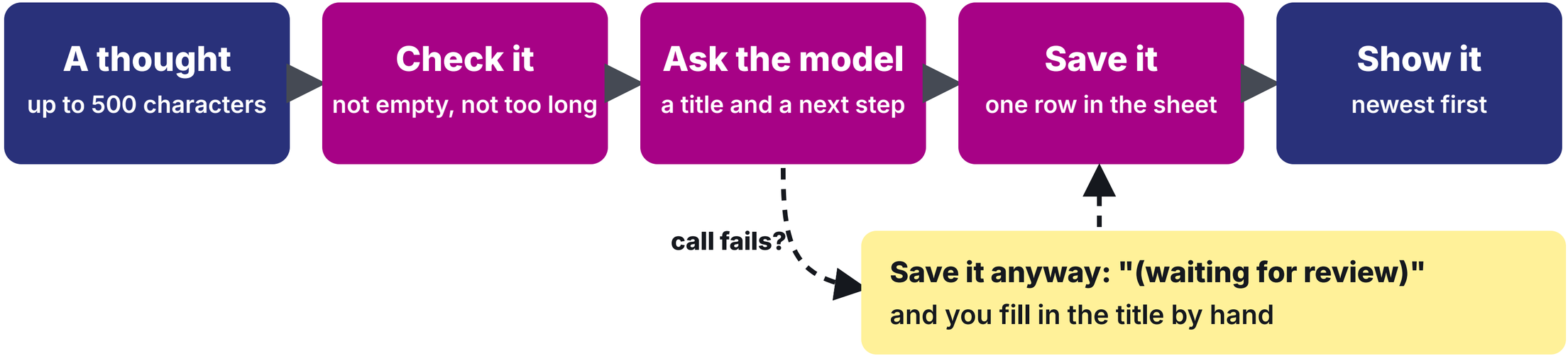
Must work: the step that creates the value. A person may do the others.



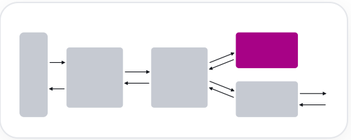
BACKEND · QUACKSYNC

Check it, ask the model, save it

And a by-hand path for when the call fails



Logic, in the backend: the steps between the input and the outcome



EVERY APPLICATION EVER · 4 DATABASE · IN A RESTAURANT

Lose the reservation book, lose the regulars

You can rebuild a kitchen. Not what the guests told you



IN A RESTAURANT

The reservation book, the regulars' usual orders, the pantry list

IN YOUR APP

The database keeps what users entered and what you need again

IN QUACKSYNC

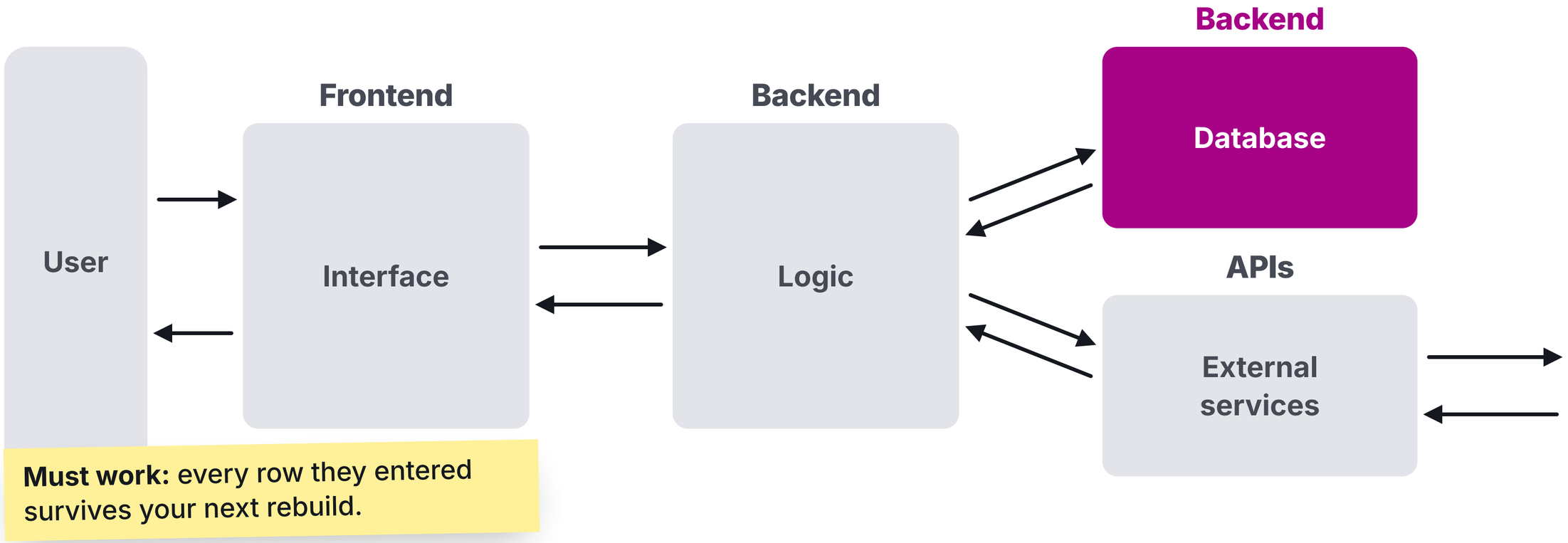
Every thought, with its title and next step

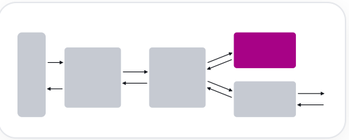
ILLUSTRATIVE A restaurant as the analogue of Every application ever, course graphic. Photo: valentin_b90 on Pixabay.

EVERY APPLICATION EVER · 4 DATABASE

Database: the one part you can't rebuild

Also called the data. What your users entered, kept safe





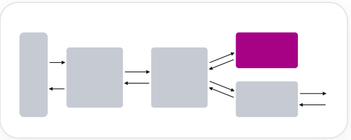
DATABASE · PROTOTYPE WITH A SPREADSHEET

A sheet is your first database

One row per thing, one column per field, fixable by hand

	A · saved	B · thought	C · title	D · next step
2	19 Oct 07:12	what if the bathroom mirror showed my calendar	Mirror calendar	Sketch it on paper
3	19 Oct 07:40	a podcast about failed inventions	Failed inventions podcast	List 5 episodes
4	20 Oct 06:55	duck that reminds me to drink water	(waiting for review)	

When the pieces start to relate to each other, a sheet becomes a database. Until then, a sheet is enough.



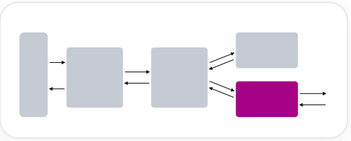
CORE PRINCIPLE

Rebuild any part, as often as you like.

Lose 0 rows they entered

The code is disposable. What users entered is not.

Who came back is what you measure next.



EVERY APPLICATION EVER · 5 EXTERNAL SERVICES · IN A RESTAURANT

Nobody bakes their own bread

Suppliers deliver through the back door, and every delivery costs



IN A RESTAURANT

The bakery delivers the bread, the card terminal takes the payment

IN YOUR APP

External services answer through an API: payments, mail, the AI model

IN QUACKSYNC

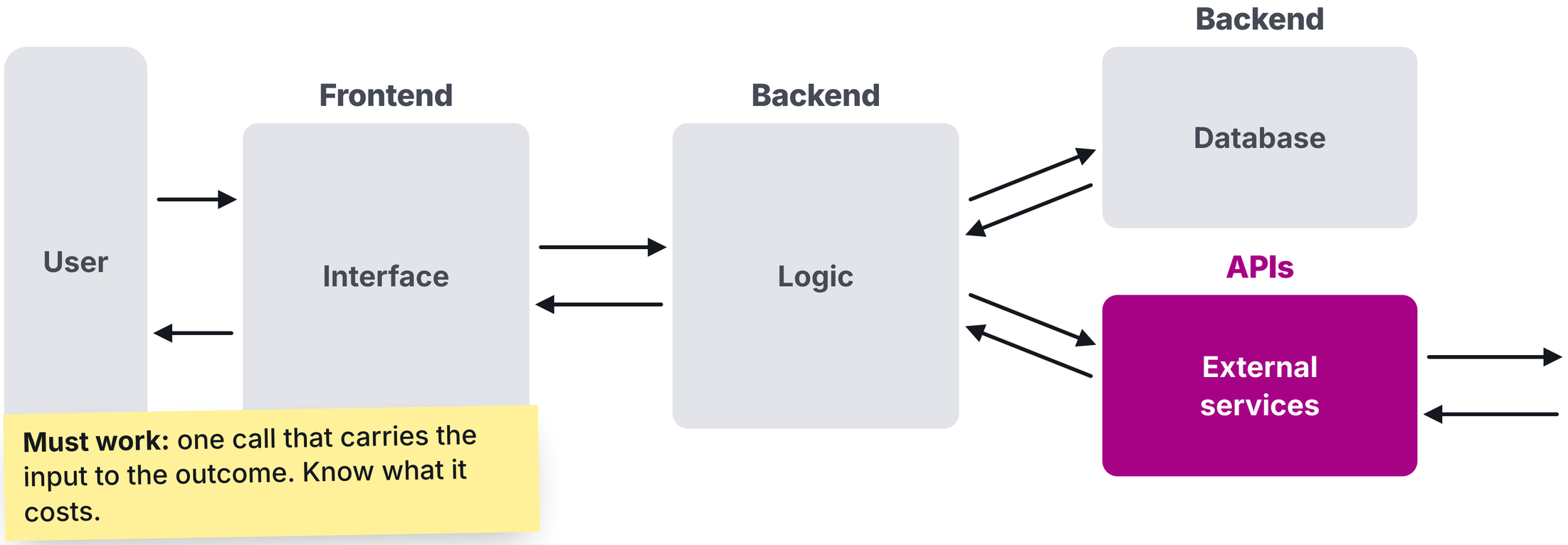
An AI model writes the title and the next step

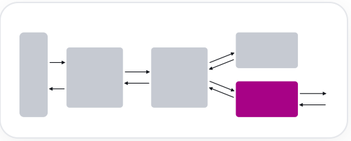
ILLUSTRATIVE A restaurant as the analogue of Every application ever, course graphic. Photo: steven_yu on Pixabay.

EVERY APPLICATION EVER · 5 EXTERNAL SERVICES

External services: they come in through APIs

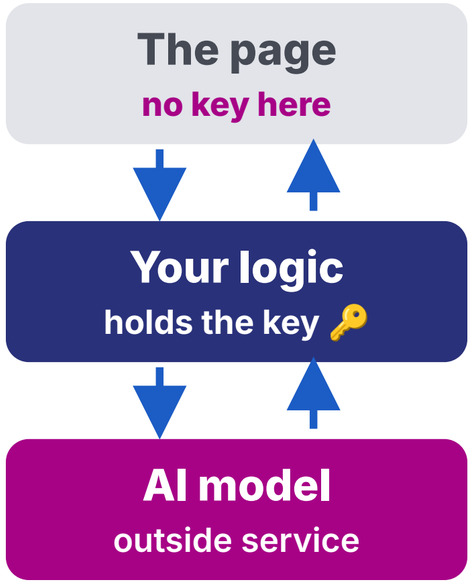
Also called APIs. Payments, mail, maps, the AI model





EXTERNAL SERVICES · ONE CALL

One call turns a thought into a next step



the thought, then the result

key + prompt, then a title and a next step

spending limit set

QuackSync

Saved · just now

YOU TYPED

“an umbrella that tells you when to leave the house”

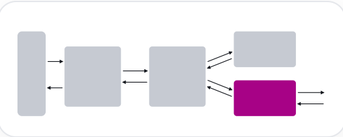
YOU GOT BACK

Leave-o-meter umbrella

NEXT STEP

Ask 3 commuters when they check the weather

ILLUSTRATIVE QuackSync is made up. Screen prototyped for this deck.



EXTERNAL SERVICES · BEFORE THE FIRST CALL

Set a limit, then hide the key

The key is a password that spends your money, and you see it only once

1 · A LIMIT THAT STOPS ALL CALLS

€0.42 of €20 used this month

Resets on the 1st of each month

Monthly budget
The most your account can spend in one month.
€20 EUR

Stop all requests when the budget is reached
Calls fail with an error until next month or until you raise the budget.

Email alert
Send a message to you@example.com when usage reaches this share of the budget.
80 % of budget

2 · THE KEY, SHOWN ONCE

Save your secret key

You will see this key only once. Store it like a password.
Never put it in a web page.

mw-sk-7f3a••••••••••••••••••c91e Copy

Done

ILLUSTRATIVE A made-up provider console, prototyped for this deck, cropped. Real consoles differ in detail.

GATE QUESTION · HANDS

Which part did your last dummy fake?

1. Interface
2. Logic
3. Database
4. External services

1:00

NOW YOU · ALONE, THEN COMPARE

Draw your MVP on the five parts

User	Who brings what, and wants what?
Interface	Where does the input go in?
Backend	What turns it into the outcome?
Database	What must you keep?
Services	Which external service?
By hand	Which part do you do yourself?

3:00

OUTPUT

Five parts, each marked: must work, or by hand

INPUT 1 OF 4 · MAP IT · THE ANSWERS

Map it: one part works, the rest by hand, zero rows lost

1

What is every app made of?

A user and four parts: interface, backend, database, external services

2

Which part must really work in a prototype?

The one that creates the value. A person may do the rest by hand

3

What must never break?

The data your users entered. Rebuild everything else

Shrink it: the cheapest version that works

Build the least that delivers, because testing is the slow part.

01 ~~Previously on Ambitious Entrepreneurship~~
02 ~~Your presentations~~
03 ~~Map it: the parts of every app~~
04 Shrink it: the cheapest version
05 Build it: you steer the agent
06 Ship it: into users' hands
07 Wrap-up and check-out
08 Workshop: build it, then hand it over
09 Case discussion

INPUT 2 OF 4 · SHRINK IT · THREE QUESTIONS

How little can you build?

1

What is the cheapest build that delivers the outcome?

2

Why not build it all, now that building is cheap?

3

Does the tool decide how big you think?

THE FREEZE RULE

The third time, you get a button

AI for the step that varies. Fixed steps for what repeats

1st time

AI does it, you check

2nd time

AI shows what is different

3rd time

fixed steps, no AI inside

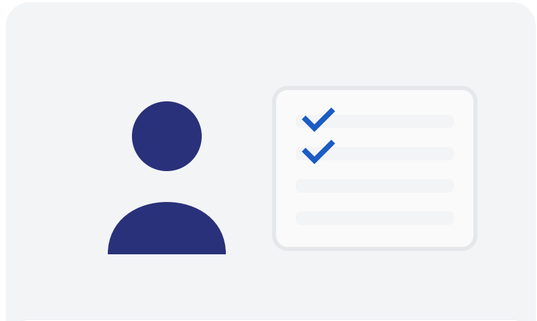
Button

Bastian Burger (2026), Towards Intent-Based Computing, unpublished whitepaper, the freeze rule.

FOUR WAYS TO BUILD · 2026

Writing code got cheap. Choosing what to build did not

Start on the left. Move right only for the part that hurts

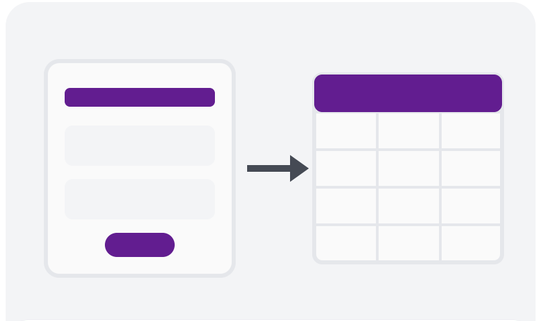


By hand

You are the backend

Fast for The first five users, today

Watch It does not scale, and that is fine for now

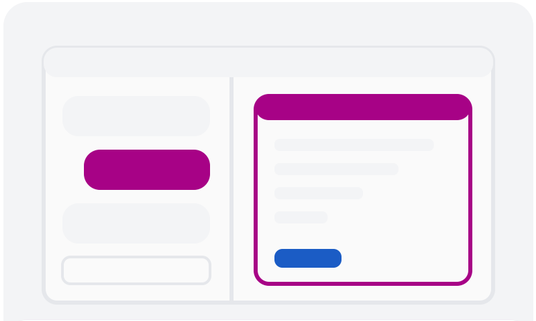


No-code tools

A form, a sheet, a flow

Fast for A workflow: form in, result out

Watch Logic spread across several tools

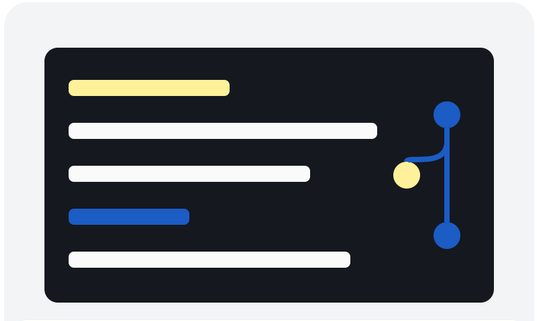


AI app builder

Prompt in, hosted app out

Fast for An app with data and hosting in an hour, for example Lovable or Replit

Watch It builds whatever you ask



AI coding agent

Your code, in your own folder

Fast for Anything, in your own code folder, for example Claude Code or Cursor

Watch You review every step

A WORD OF CAUTION

Building got fast. Testing did not

So founders overbuild, and trust it before anyone used it

Before AI tools
building was the slow part



With AI tools
testing is the slow part



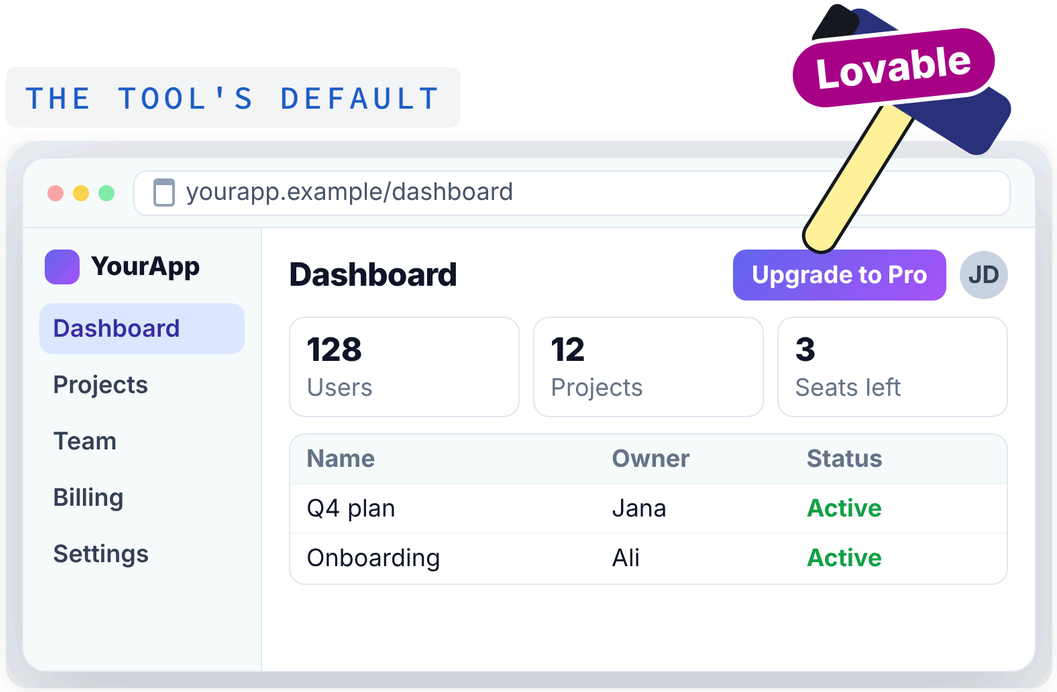
The risk: overbuilding, and trusting a build nobody has used

THINK BIG, PROTOTYPE SMALL

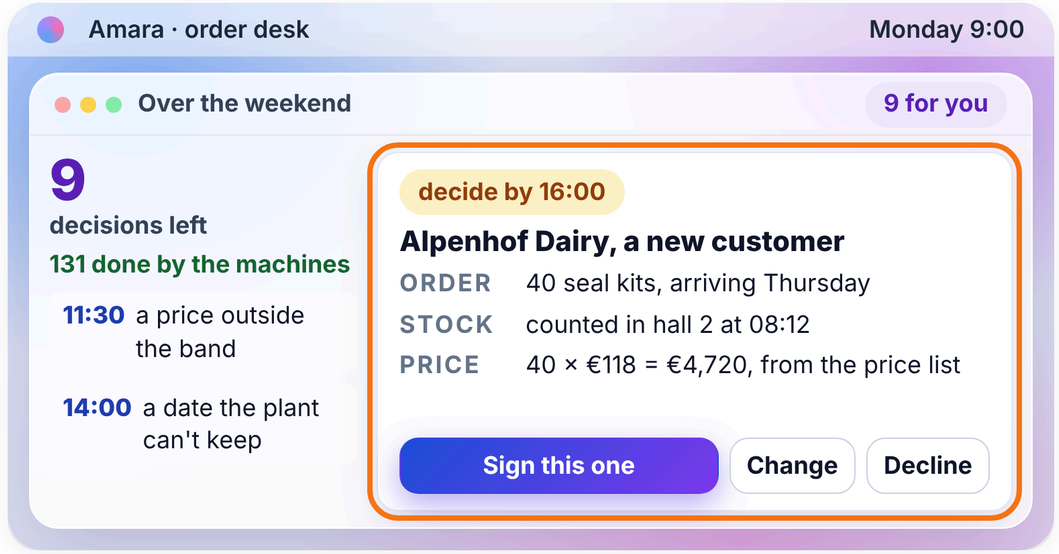
If all you have is a Lovable, everything looks like a SaaS

Keep the five-year picture. Prototype one slice of it today, and do the rest by hand

THE TOOL'S DEFAULT



YOUR FIVE-YEAR PICTURE



ILLUSTRATIVE Both screens prototyped for this deck, the five-year picture after Finding ideas (M02). Names are trademarks of their owners.

NOW YOU · IN YOUR TEAM

What is the cheapest version that delivers?

Outcome What must the user get, once?

By hand Which part do you do yourself for now?

Build A tool, a builder or an agent?

Sheet Is a sheet enough for the data?

2:00

OUTPUT

One sentence: form, sheet, call, by hand

INPUT 2 OF 4 · SHRINK IT · THE ANSWERS

Shrink it: the smallest build that delivers

1

What is the cheapest build that delivers the outcome?

Often a form, a sheet and you. Move right only for the part that hurts

2

Why not build it all, now that building is cheap?

Testing is the slow part. Every extra feature delays it and inflates your confidence

3

Does the tool decide how big you think?

Only if you let it. Keep the five-year picture, build one slice of it

INPUT 3 OF 4 · CHAPTER 5 OF 9

Build it: you are the product owner

Steer an AI agent in small checked steps, in code or in a builder.

01 ~~Previously on Ambitious Entrepreneurship~~
02 ~~Your presentations~~
03 ~~Map it: the parts of every app~~
04 ~~Shrink it: the cheapest version~~
05 **Build it: you steer the agent**
06 **Ship it: into users' hands**
07 Wrap-up and check-out
08 Workshop: build it, then hand it over
09 Case discussion

INPUT 3 OF 4 · BUILD IT · THREE QUESTIONS

Who steers when an agent builds?

1

Why do AI builds run away from you?

2

How do you steer one without reading code?

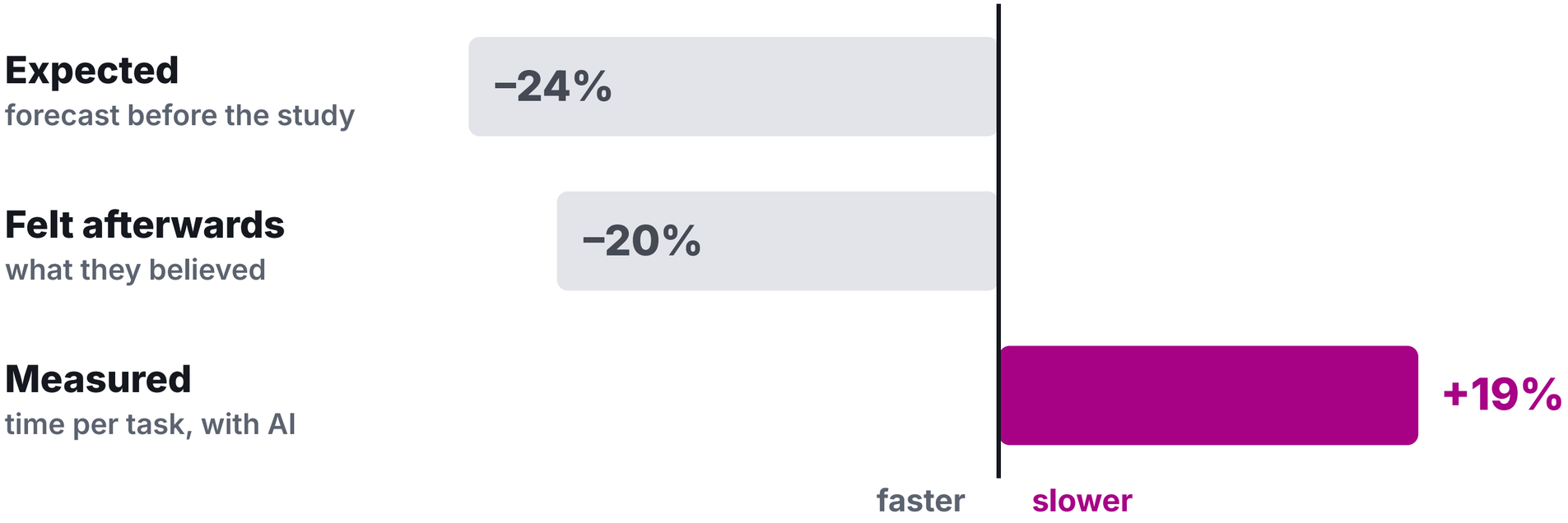
3

Coding agent or app builder?

WHY IT MATTERS

It feels faster. Measure it

16 experienced developers, their own codebases, 246 tasks, early-2025 tools

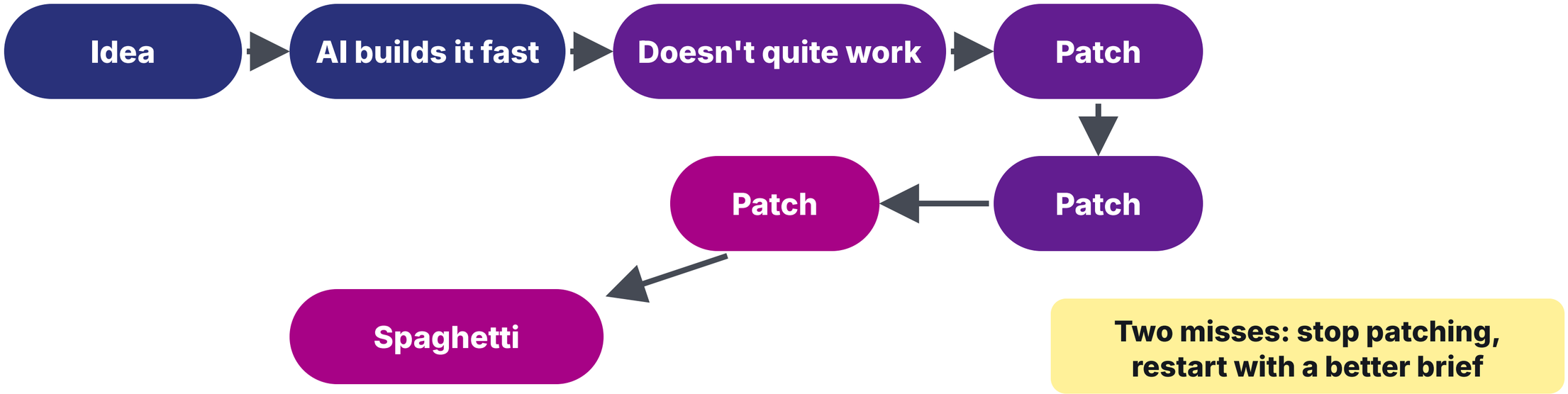


RESEARCH METR (2025), Measuring the impact of early-2025 AI on experienced open-source developer productivity, randomized controlled trial, 10 July 2025.

THE PATCH SPIRAL

Patch, patch, patch, spaghetti

AI removed the bottleneck of building, not the bottleneck of deciding



GATE QUESTION · HANDS

Who had an AI fix one thing and break another?

1:00

CORE PRINCIPLE

You are the product owner. The agent is the contractor

It builds whatever you ask, and it never pushes back on scope

FROM

"Build me a recruiting platform." The agent decides what goes in.

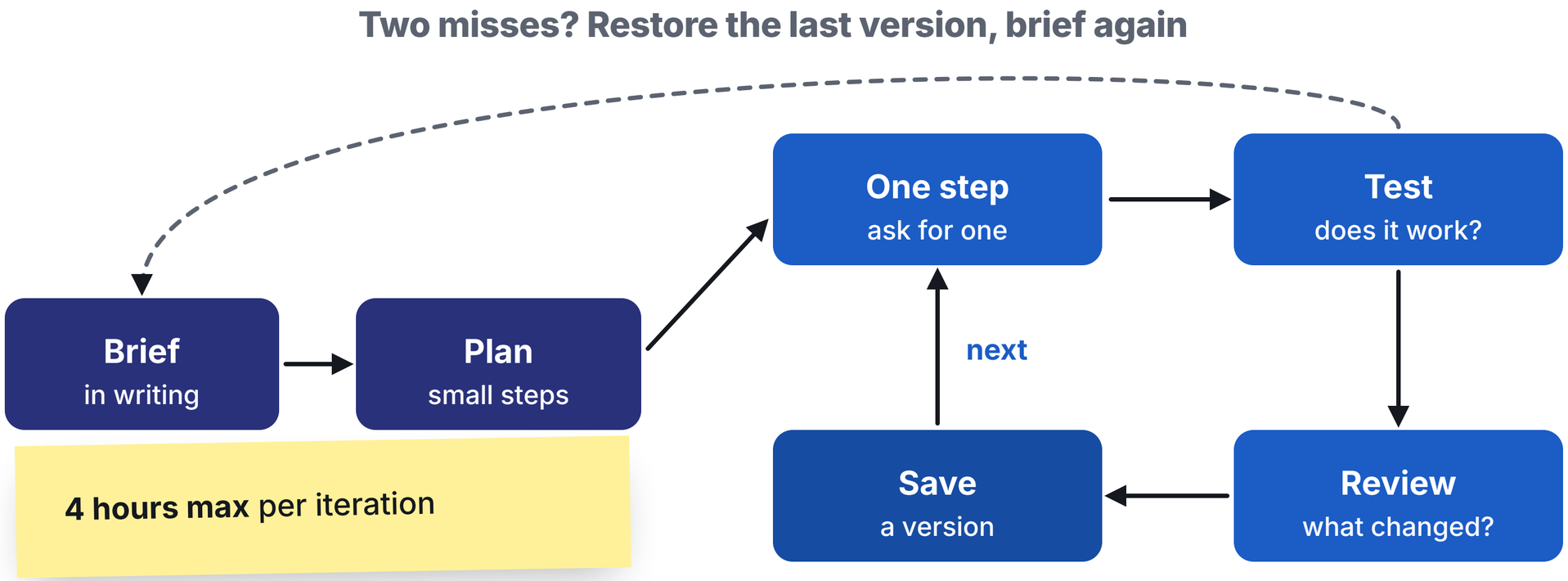
TO

You decide what and why. The agent proposes how. You check it.

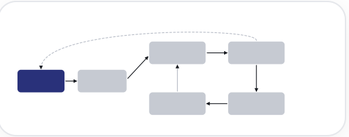
> Never let the contractor decide what to build.

THE AGENT LOOP

Brief, one step, test, review, save



FRAMEWORK Claude Code docs, "Best practices for Claude Code" (explore, plan, implement, commit, and after two failed corrections start fresh with a better prompt), accessed 2026-10-19. Two-miss rule also in the course resource Best Practices: Coding with AI.



MICRO CASE · QUACKSYNC · BUILD A, A CODING AGENT · STEP 1 OF 6

Write the brief once

One page, eight headings. Four of them do most of the work.

PROJECT_INSTRUCTIONS.md · one page

```
## Who it is for
## The one feature
## Input and outcome
## The parts
## What stays manual
## Do not build
## Tech
## How we work
```

- WHO IT IS FOR

  “People who get their best ideas in the shower”

- THE ONE FEATURE

  “Type a raw thought into one box.”

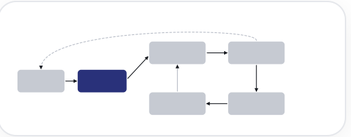
- WHAT STAYS MANUAL

  “save the thought anyway with the title "(waiting for review)"”

- DO NOT BUILD

  “Accounts, login, editing, deleting, voice input, sharing, settings …”

A real build by an AI coding agent (Claude) from this brief, 19 October 2026, records in the course workspace. QuackSync is a made-up venture.



MICRO CASE · QUACKSYNC · BUILD A, A CODING AGENT · STEP 2 OF 6

Ask for a plan, not code

Each step small enough to test. No code before the plan.

PLAN.md as first committed · the steps, then step 1 in full

```
## Step 1: Save and list thoughts in a CSV file
## Step 2: Ask the AI model for a title and a next step
## Step 3: Save a thought, with the manual fallback
## Step 4: Render the page
## Step 5: Serve it over HTTP

## Step 1: Save and list thoughts in a CSV file
Adds: `thoughts.csv` storage. Check the input (not empty, at most 500 characters), append a thought with its title
and next step, read all thoughts back newest first.
Test that proves it: a saved thought comes back unchanged (commas, quotes and line breaks included), the newest
comes first, a missing file gives an empty list, an empty or 501-character input is refused and exactly 500 is
accepted.
```

A real build by an AI coding agent (Claude) from this brief, 19 October 2026, records in the course workspace. QuackSync is a made-up venture.

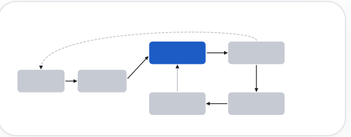
NOW YOU · 1 MINUTE

Which step would you cut or split?

The product owner's test: could I try this step myself in five minutes?

1. Save and list thoughts in a CSV file
2. Ask the AI model for a title and a next step
3. Save a thought, with the manual fallback
4. Render the page
5. Serve it over HTTP

1:00



MICRO CASE · QUACKSYNC · BUILD A, A CODING AGENT · STEP 3 OF 6

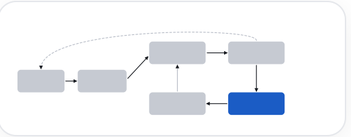
One step, one test, one version

Look at two lines: which files changed, and "OK" after the tests. A file you didn't ask for? Ask why, or undo.

$ git show --stat · $ python3 -m unittest

```
$ git show --stat b7c65d0
b7c65d0 Step 3: add submit_thought, saves the model's title and next step or the '(waiting for review)' fallback

 quacksync.py      | 18 ++++++++++++++++
 test_quacksync.py | 64 ++++++++++++++++
```



MICRO CASE · QUACKSYNC · BUILD A, A CODING AGENT · STEP 4 OF 6

The review caught what the tests didn't

A second agent found that any website could make your browser use it and spend its AI credit. Each finding became one small step.

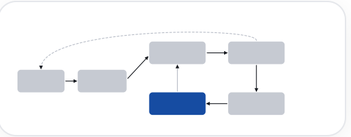
Review findings and fixes

```
## Step 6: Reject foreign Host and cross-site POSTs (review step)
## Step 7: Require a form token, fail closed (second review step)
## Step 8: Enforce localhost-only binding and say so in a README (third review step)
Checked before building: the finding does not match this code. The app reads one environment variable,
`ANTHROPIC_API_KEY`, and `main()` passes the literal `127.0.0.1` to the server. Started with `HOST=0.0.0.0
BIND=0.0.0.0`, the process still listens on `127.0.0.1` only (saved in `runs/step8-premise-check.txt`). No commit
ever read a `HOST` variable.

$ git log --oneline (review steps)
691e1d0 Reject foreign Host and cross-site POSTs (review finding)
584ff72 Require a form token, fail closed (second review)
905b57a Enforce localhost-only binding and add README (third review)

$ python3 -m unittest -v
...
Ran 59 tests in 11.146s
OK
```

A real security review of the build above, 19 October 2026. The fix is the build's own commit.



MICRO CASE · QUACKSYNC · BUILD A, A CODING AGENT · STEP 5 OF 6

Every step is a saved version

Two misses on a step? Go back to the last version and brief again.

```
$ git log --oneline --reverse

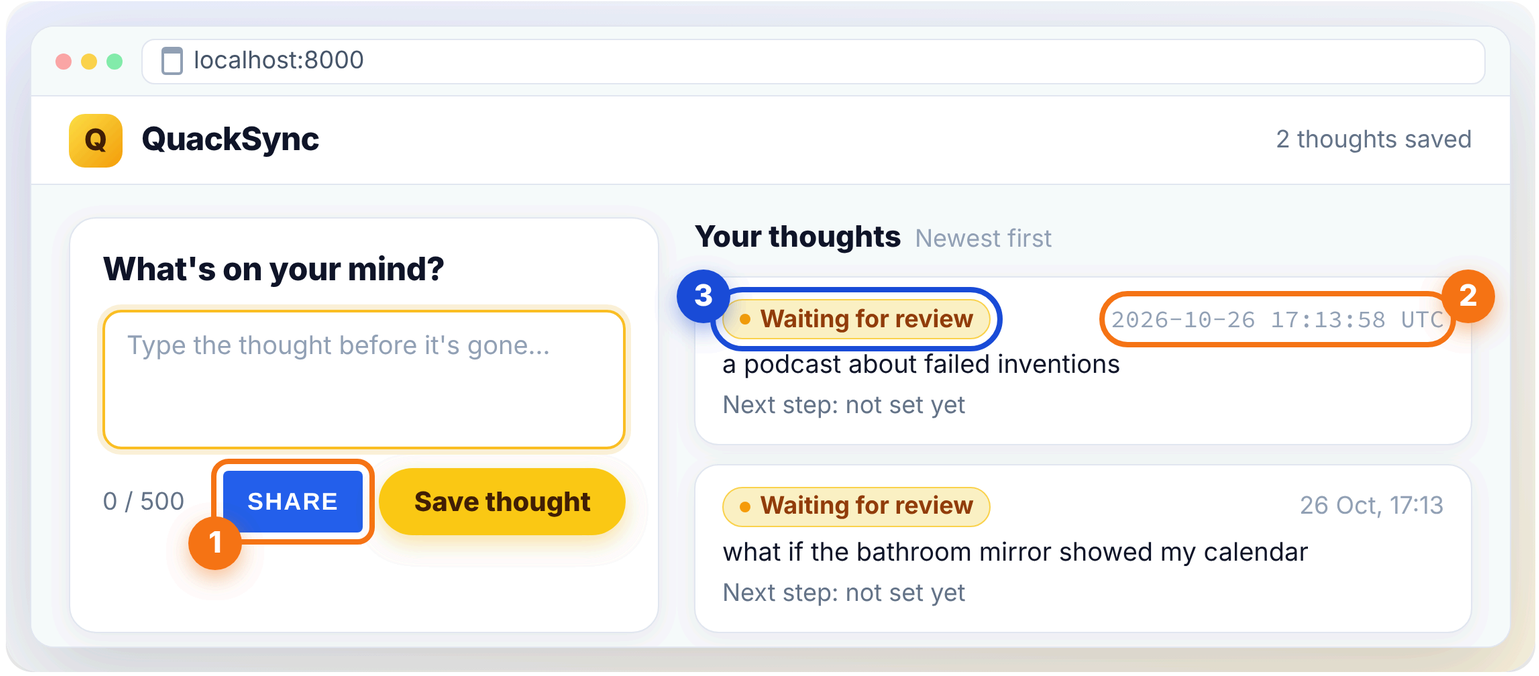
3e276a0 Add project instructions
1a93994 Add plan in small steps
4fba017 Step 1: save thoughts to thoughts.csv and list them newest first, refuse empty or over-500-
character input
6503d72 Step 2: add ask_model, a Messages API call with urllib that returns a title and a next step,
tested with a stub
b7c65d0 Step 3: add submit_thought, saves the model's title and next step or the '(waiting for review)'
fallback
0ffc73f Step 4: add render_page, one plain HTML page with the form and the newest-first list, user text
escaped
4aa29a5 Step 5: add the local web server (GET / shows the page, POST /add saves and redirects) and the
command line start
8f44927 Add review step to plan: reject foreign Host and cross-site POSTs
691e1d0 Reject foreign Host and cross-site POSTs (review finding)
a34e1b2 Add second review step to plan: form token, fail closed
584ff72 Require a form token, fail closed (second review)
a75df3a Add third review step to plan: enforce localhost-only binding (finding's HOST premise not found
in code)
905b57a Enforce localhost-only binding and add README (third review)
```

A real build by an AI coding agent (Claude) from this brief, 19 October 2026, records in the course workspace. QuackSync is a made-up venture.

MICRO CASE · QUACKSYNC · BUILD A, A CODING AGENT · STEP 6 OF 6

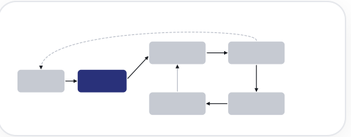
Styled in minutes. Now click every button

It looks finished. Check what works before a user does



1. **Share does nothing,** and the brief said no sharing
2. **Two date formats,** two button styles
3. **No key, so both wait for review.** The fallback works

ILLUSTRATIVE Screen prototyped for this deck, as a coding agent styles a first version. The two thoughts and the missing key are from the real build A (Claude, 19 October 2026), whose brief asked for plain HTML. The flaws are made up. QuackSync is made up.



MICRO CASE · QUACKSYNC · BUILD B, AN AI APP BUILDER

Same brief, an app builder: plan first

Plan in 37 seconds, four steps, because the brief asked for it

.lovable/plan.md · as the builder wrote it, 37 seconds after the brief

```
## Step 1 — The page (no saving yet)
- Replace the blank home page with a simple, readable layout: title "QuackSync", one text box (500-character limit with a live counter), a Save button, and an empty list area.

## Step 2 — The database
- Turn on Lovable Cloud and create one table `thoughts`: saved time, thought, title, next step.

## Step 3 — Save and list (no AI yet)
- Save button stores the thought with title "(waiting for review)" and empty next step.

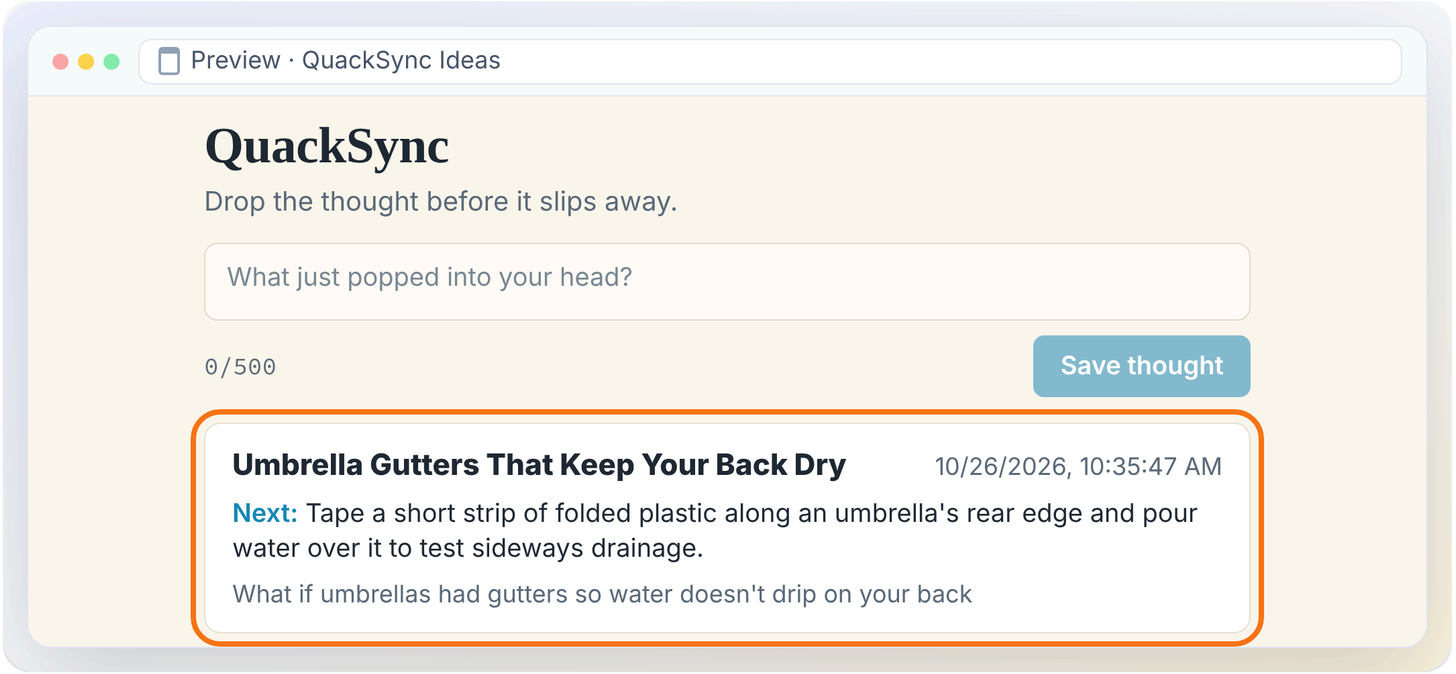
## Step 4 — AI title and next step
- On save, the server asks the built-in AI for a one-line title and one next step, then saves everything together.
```

A real build in Lovable from the same brief, 26 October 2026, records in the course workspace. QuackSync is a made-up venture.

MICRO CASE · QUACKSYNC · BUILD B, AN AI APP BUILDER

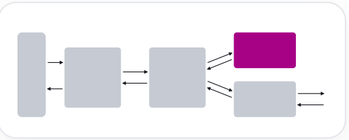
Three minutes later, the page worked

After the go-ahead the AI call worked in its own test, with no key to set



- A title and one next step from the builder's own AI
- Styled without asking: a serif title, a cream page

ILLUSTRATIVE A real build in Lovable from the same brief, 26 October 2026, records in the course workspace. QuackSync is a made-up venture. Screen redrawn sharp from Lovable's own screenshot, same words and layout.



MICRO CASE · QUACKSYNC · BUILD B, AN AI APP BUILDER

A real database, with rules worth reading

You need not read the code. Read who may read, add, change or delete

Table thoughts · the row its own test saved, read on 26 October 2026

`created_at (UTC)`	2026-10-26 10:35
`thought`	What if umbrellas had gutters so water doesn't drip on your back
`title`	**Umbrella Gutters That Keep Your Back Dry**
`next_step`	Tape a short strip of folded plastic along an umbrella's rear edge and pour water over it to test sideways drainage.

The rules the builder wrote into the database

```
CREATE POLICY "Anyone can read thoughts" FOR SELECT USING (true);
CREATE POLICY "Anyone can add thoughts" FOR INSERT WITH CHECK (true);
```

No rule to update or delete: the page cannot erase a thought.

Good: no one can erase a thought.

Check: every visitor reads every thought. Before strangers, ask the builder to show each visitor only their own.

A real build in Lovable from the same brief, 26 October 2026, records in the course workspace. QuackSync is a made-up venture.

MICRO CASE · QUACKSYNC · TWO BUILDS COMPARED

Both took minutes. They differ in what you own

And in what is left for you to check

	BUILD A · A CODING AGENT	BUILD B · AN APP BUILDER
Where it runs	Only on the laptop that built it	In the builder's cloud, behind a sign-in until you publish
Data	A CSV file next to the code	A database table with rules
The AI call	Never ran: faked in the tests, no key set	Worked in its own test, the builder keeps the key
Checks	59 tests, three security reviews by a second agent	3 tests and one real save, no review asked for
First version that ran	3 minutes after the plan, by its saved versions	3 minutes after the go-ahead, 2 Lovable credits
You own	Every line, plain Python	A hosted project, code exportable

DATA Build A, 19 October 2026, and build B, 26 October 2026, from the same brief, records in the course workspace. QuackSync is made up.

SPEED MATTERS

One iteration should not take more than

4 hours

One iteration runs from the brief to a version a user can try. Longer means it was too big.

Steps inside it take minutes.

NOW YOU · ALONE

Write the first lines of your brief

Who — Who is it for?

Feature — The one feature, input and outcome

Manual — What stays by hand?

Not — What must it not build?

3:00

OUTPUT

Four lines your agent can read

INPUT 3 OF 4 · BUILD IT · THE ANSWERS

Build it: you own the what, the agent the how

1

Why do AI builds run away from you?

The agent builds whatever you ask, and fast feels like progress

2

How do you steer one without reading code?

Brief once, plan first, one step, test, review, save. Two misses: restart

3

Coding agent or app builder?

Both take minutes. Pick by what you must own and check

Ship it: into users' hands

Put it in front of your segment, watch without helping, fix what stopped them.

01 ~~Previously on Ambitious Entrepreneurship~~
02 ~~Your presentations~~
03 ~~Map it: the parts of every app~~
04 ~~Shrink it: the cheapest version~~
05 ~~Build it: you steer the agent~~
06 **Ship it: into users' hands**
07 Wrap-up and check-out
08 Workshop: build it, then hand it over
09 Case discussion

INPUT 4 OF 4 · SHIP IT · THREE QUESTIONS

When does a build become evidence?

1

Who counts as a user?

2

How do you know it works?

3

What do you do before you build again?

FROM YOUR LAPTOP TO THEIR HANDS

Live means a stranger can open it

Four things before you send the link

A link
opens on their phone

Its own data
each user sees only theirs

Keys in secrets
with a spending limit

Imprint, privacy
say which AI sees the data

App builder: its publish button. Coded build: a host and a managed database

> **If you're not embarrassed by the first version of your product, you've launched too late.**

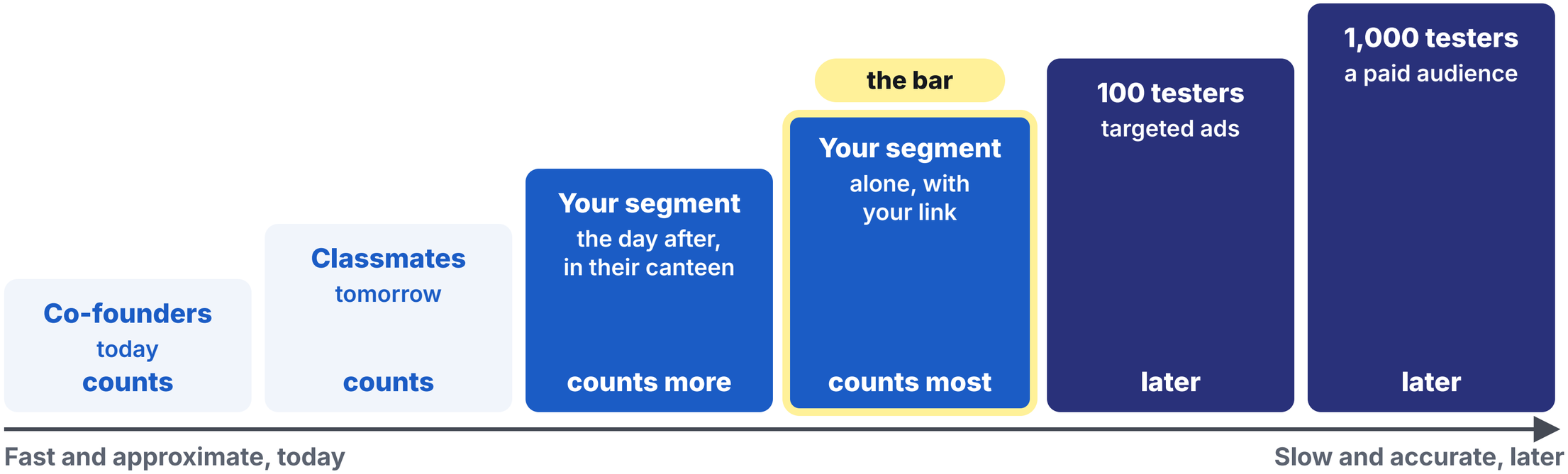
Reid Hoffman, co-founder of LinkedIn

Hoffman, R. (2017, 29 March). If there aren't any typos in this essay, we launched too late! LinkedIn.

TESTING YOUR PROTOTYPE

Test every day. Everyone counts

Slow and accurate counts most, and the bar counts your segment

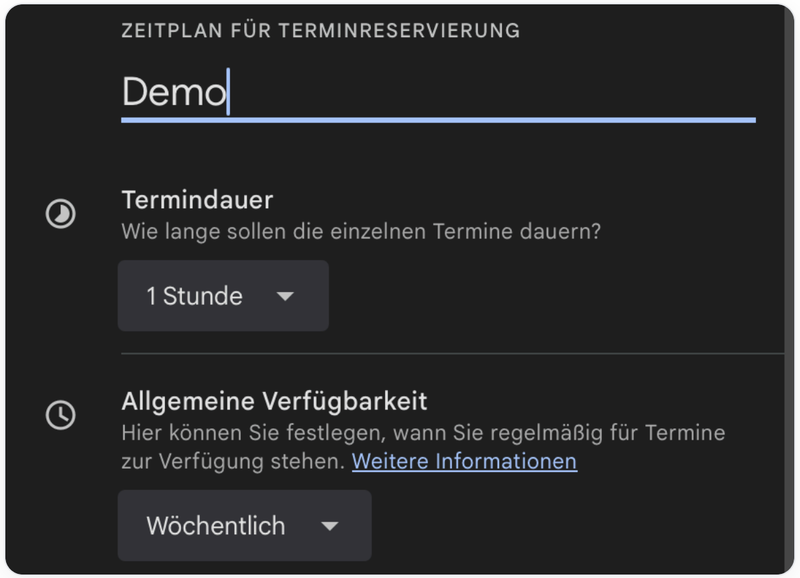


WITHOUT YOUR HELP

Let them use it alone

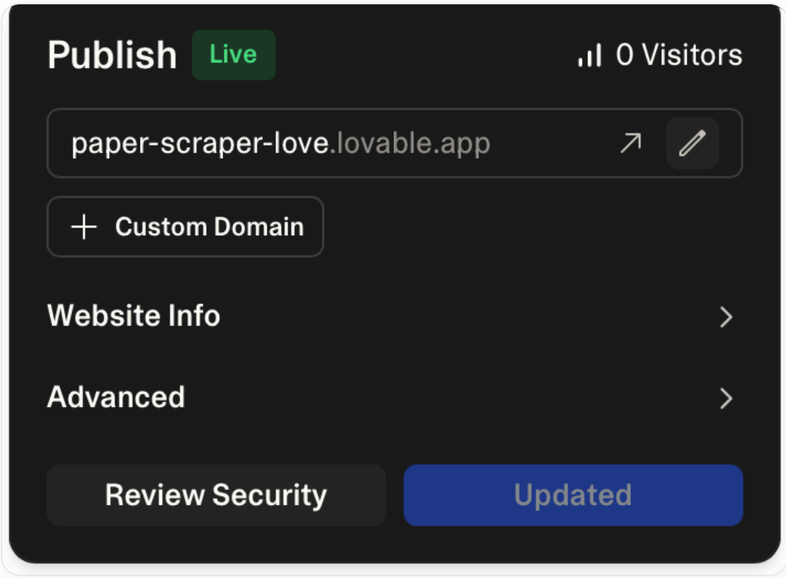
One hint and it is a guided demo again

GUIDED · YOU STEER



Solution space (M05): you book the call

ALONE · ONE LINK

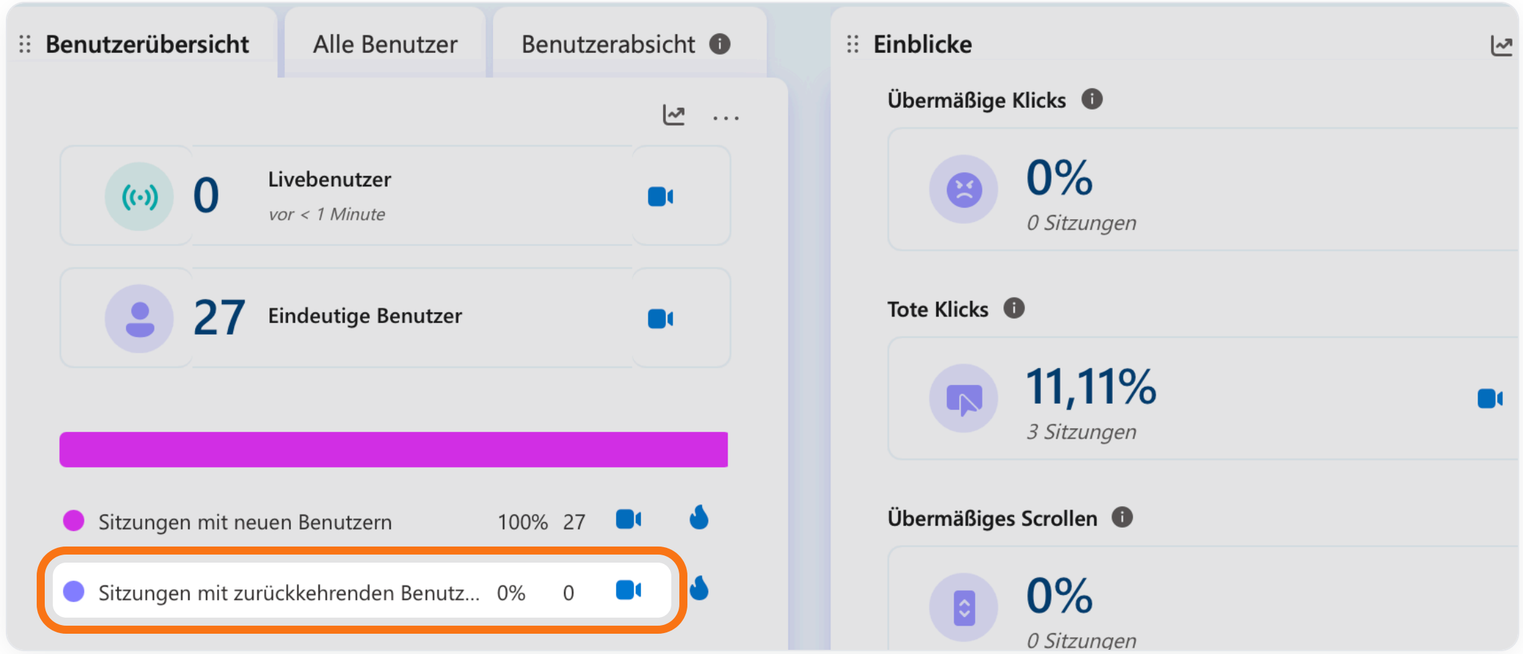


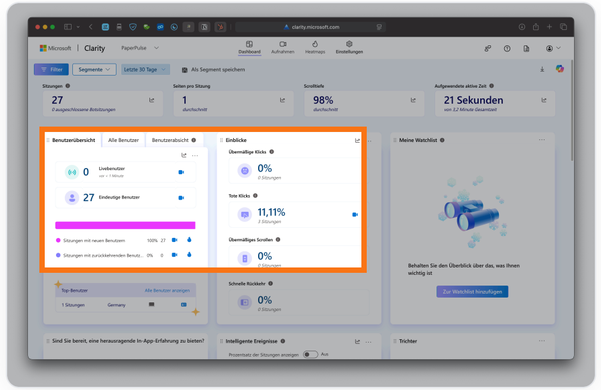
From today: they decide what to do

WATCH WHAT THEY DO

Who came back, and where they got stuck

Returning users, dead clicks and a recording per session





Ask before you record. Consent covers course staff.

TWO QUESTIONS PER USER

Did they reach the outcome? Did they come back?

Reached the outcome?

the value arrived, without your help

Came back?

used it again, unprompted

User	Without help	Outcome	Came back	What stopped them
Recruiter, agency	yes	yes	day 3	–
Founder, B2B tool	yes	no	no	did not find "Add"

ILLUSTRATIVE Use record per user, made up. Framework 03 questions 4 and 8 read it in The kill decision (M07).

BEFORE EACH ITERATION

Four questions before you build again

1 **Does it match the problem?**
the problem you confirmed, in their words

2 **For whom is it 10x better?**
than what they use today

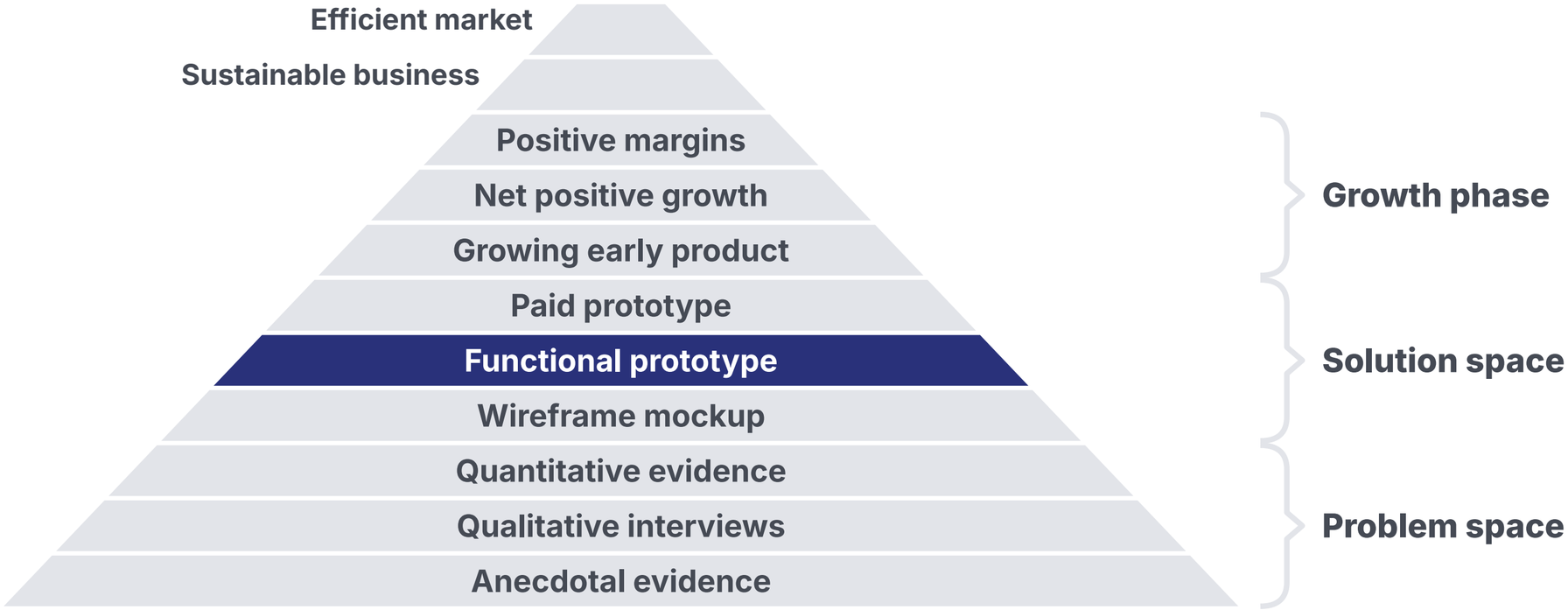
3 **Can you build it in 4 hours?**
or is the iteration too big

4 **How does it reach their hands?**
a link, today, without you

WHERE YOU STAND

A slice that works is signal until someone pays

Your MVP is built to be paid for. Until someone pays, it is a functional prototype



DON'T STRUGGLE ALONE

One leads the build. Everyone brings users

One founder leads the build

briefs the agent

one step at a time

ships the fixes

and brings users too

Everyone brings users

invites first buyers

watches, never steers

logs one row per user

NOW YOU · IN YOUR TEAM

Who gets the link today?

First Who said yes to your last ask?

Then Which sign-ups and interviewees?

Watch How do you watch without steering?

2:00

OUTPUT

Five names and a way to watch

INPUT 4 OF 4 · SHIP IT · THE ANSWERS

Ship it: used, not clicked

1

Who counts as a user?

Everyone you test with, every day. The bar counts your segment

2

How do you know it works?

They reach the outcome without your help, and come back

3

What do you do before you build again?

Fix what stopped them, cut what nobody used, ask the four questions

CHAPTER 7 OF 9

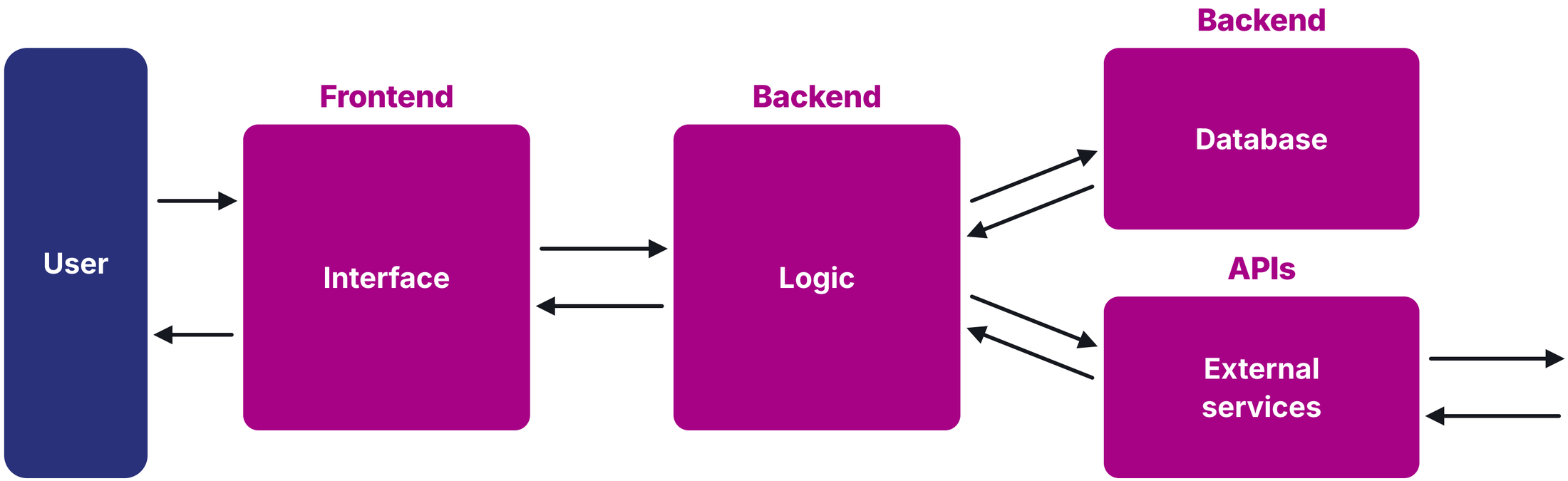
Wrap-up and check-out

Map it, shrink it, build it, ship it, in four pictures and three questions.

01 ~~Previously on Ambitious Entrepreneurship~~
02 ~~Your presentations~~
03 ~~Map it: the parts of every app~~
04 ~~Shrink it: the cheapest version~~
05 ~~Build it: you steer the agent~~
06 ~~Ship it: into users' hands~~
07 **Wrap-up and check-out**
08 Workshop: build it, then hand it over
09 Case discussion

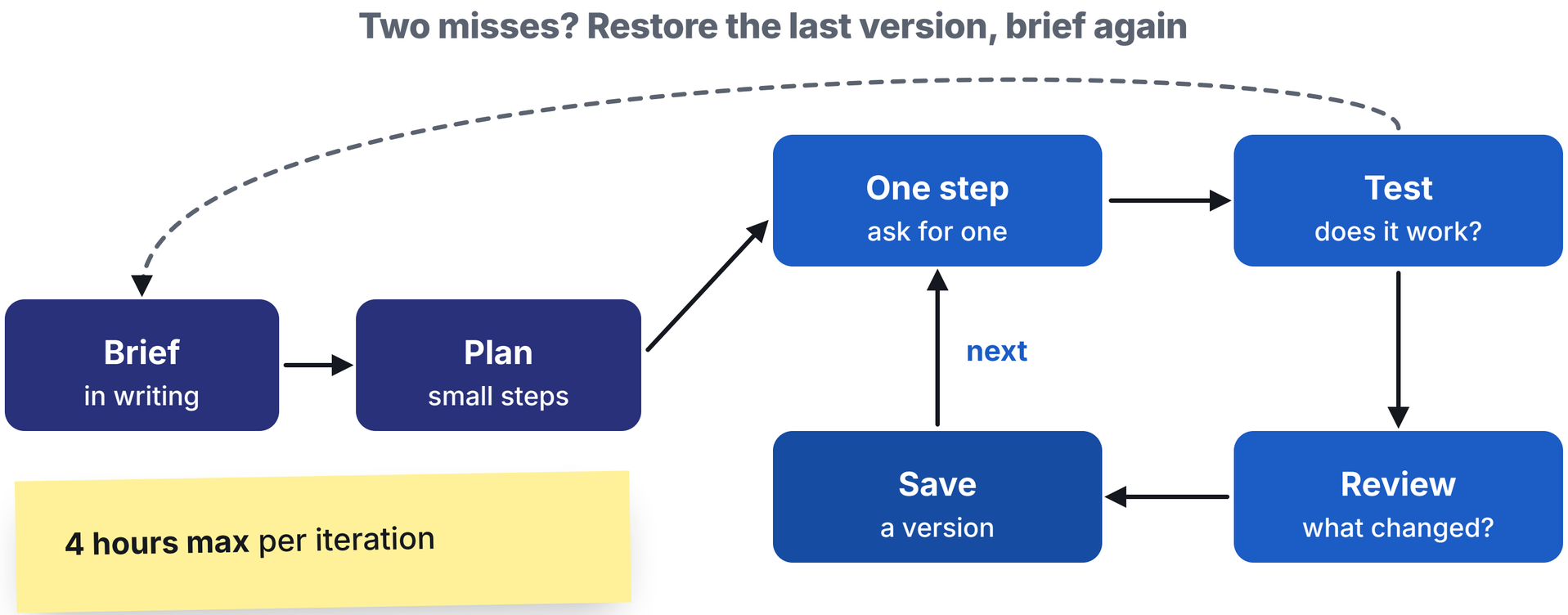
WRAP-UP · MAP IT

Five parts: must work, or by hand for now



WRAP-UP · BUILD IT

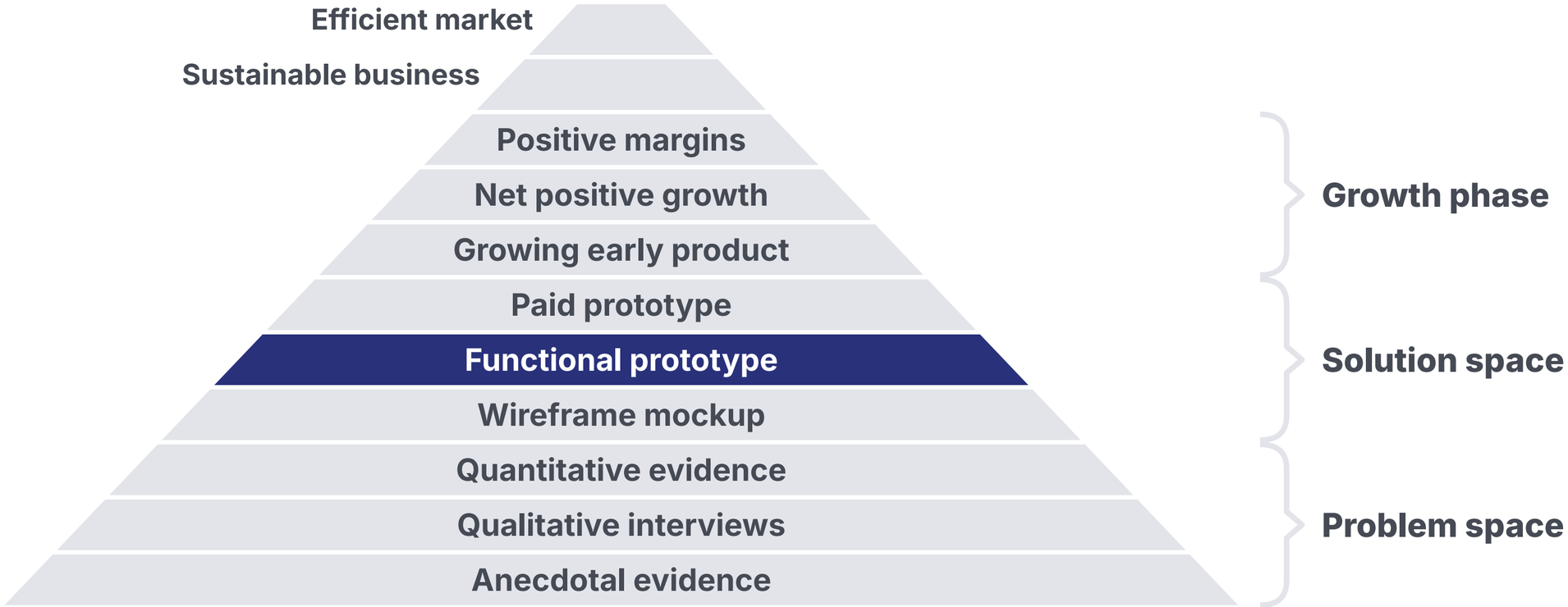
The agent loop



FRAMEWORK Claude Code docs, best practices, and the course resource Best Practices: Coding with AI.

WRAP-UP · SHIP IT

Rung five, signal until someone pays



REFLECTION 1 OF 3 · THE CONTENT

The Content – What could you apply



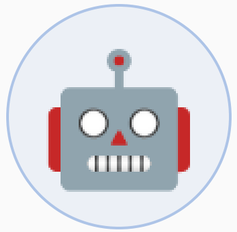
What could you apply?



What gave you a hard time?

REFLECTION 2 OF 3 · THE PROCESS

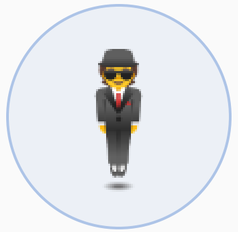
The Process – Man vs. Machine



Where did AI actually help you?



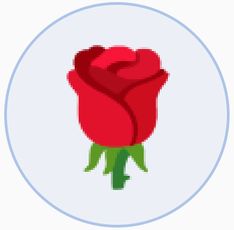
Where was your genius needed?



What are your best hacks to share?

REFLECTION 3 OF 3 · THE RESULTS

Rose, thorn, bud



What was the breakthrough today?



What was the hardest part today?



What is an idea/feature you didn't have time to finish, but want to explore later?

HOMEWORK UNTIL NEXT TIME

Put your MVP live, let 10 people per student from your segment use it without your help, and ship 2 fixes for what stopped them.

10 users each

The learning floor is a live MVP per team, 5 such users per student and 1 fix.

Your line: fewer than 3 of your first 10 users reach the outcome by Module 07, and you rebuild the feature. Nobody comes back unprompted, and you bring a pivot.

HOMEWORK · HOW TO DO IT

Hand it over. Watch. Fix what stopped them

1. **Split the work.** One leads the build, everyone brings users.
2. **Invite first buyers,** then sign-ups, then your segment's groups.
3. **Watch without steering.** Mute yourself or watch a recording.

! **Watch out.** One hint makes it a guided demo.

4. **Ask four questions before each build.** Does it match the problem? For whom is it 10x better? Can you build it in 4 hours? How does it reach them?
5. **Fix what stopped them, cut what nobody used.** Two misses? Restart from your last version.

6. **Keep their data.** Never wipe it. Collect only what you need, and say where it goes, AI included (Legal Basics).
7. **Keep the ask.** Show your price. An early payment counts.
8. **Get a blind second opinion.** Give an AI your raw records, minus hypothesis and names. Note where it disagrees.

How to do it, from the Prototyping (M06) homework. Legal Basics.

HOMEWORK · WHAT YOU COME BACK WITH

Good is used, not clicked

	GOOD	WEAK
MVP	One feature, real input to outcome, 23 versions	Five features, "almost works"
USERS	31 recruiters without help, 14 got their nudge, 6 came back	12 friends clicked
FIXES	"Nobody found Add." Moved, 3 more reached it	Dark mode

ILLUSTRATIVE What you come back with, Prototyping (M06) homework. The examples follow the made-up recruiting venture.

Sources in this deck

How to read the source lines. Research: a published study. Data: dated records. Framework: a published method or guide. Bastian: the course's own material. Illustrative: a made-up example.

Best Practices: Coding with AI. Course resource, resources/Best_Practices_Coding_with_AI.pdf.

Anthropic. (n.d.). *Best practices for Claude Code*. Claude Code Docs. https://code.claude.com/docs/en/best-practices, accessed 2026-10-19.

Becker, J., Rush, N., Barnes, E., & Rein, D. (2025). *Measuring the impact of early-2025 AI on experienced open-source developer productivity*. METR, 10 July 2025. https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/, accessed 2026-10-19.

Hess, A. (2013, 14 August). *The oldest McDonald's as architecture*. Alan Hess on Architecture. https://alanhess.blogspot.com/2013/08/the-oldest-mcdonalds-as-architecture.html, accessed 2026-10-19.

Sources in this deck

Hoffman, R. (2017, 29 March). *If there aren't any typos in this essay, we launched too late!* LinkedIn. https://www.linkedin.com/pulse/arent-any-typos-essay-we-launched-too-late-reid-hoffman/, accessed 2026-10-19.

Ambitious Entrepreneurship. Framework 02, Validation Hierarchy, the Glossary and the course resource Best Practices: Coding with AI. Course files.

Graham, P. (2013, July). *Do things that don't scale*. paulgraham.com/ds.html.

Veracode. (2025). *2025 GenAI Code Security Report*. Vendor report, quoted in the talk track of the review slide.

Ries, E. (2011). *The Lean Startup*. Crown Business.

Design Council. (2004). *The double diamond*, as drawn in the course.

See you next time!

MODULE 06 · PROTOTYPING WITH AI · WORKSHOP · 90 MINUTES

Workshop: Build it, then hand it over

Map the parts, build in small steps, and send the link before you leave.

01 Previously on Ambitious Entrepreneurship
02 Your presentations
03 **Map it: the parts of every app**
04 **Shrink it: the cheapest version**
05 **Build it: you steer the agent**
06 **Ship it: into users' hands**
07 Wrap-up and check-out
08 **Workshop: build it, then hand it over**
09 Case discussion

WORKSHOP · STEP 1 OF 3

Map the parts

1 STEP 1 · 20 MIN

MAP THE PARTS

Know what must work.

→ Draw your MVP: interface, logic, database, outside services.
→ Mark each part: must work, or by hand for now.
→ Would a form and a sheet do?
→ Write the brief: user, feature, input, outcome, manual, not-to-build.

Output A map and a brief.

Minutes 20

20:00

WORKSHOP · STEP 2 OF 3

Build in small steps

2 STEP 2 · 50 MIN

BUILD IN SMALL STEPS

Real input runs through to the outcome.

→ Load the brief as project instructions. Plan first, no code.
→ One step at a time: ask, read the change, test, save.
→ Two misses on one step? Restore and brief again.
→ Wire the one API call. Keep the key out of the interface.
→ Publish on your tool's hosting. Point your page's button at it.

Output A live link, versions saved.

Minutes 50

50:00

WORKSHOP · WHERE THINGS LIVE IN YOUR TOOL

Same steps, different buttons

New tonight? Take your own idea: one user, one feature, the four parts

	APP BUILDER, FOR EXAMPLE LOVABLE	CODING AGENT, FOR EXAMPLE CLAUDE CODE
Brief	Project knowledge, in its settings	A CLAUDE.md file in your folder
Save a version	Automatic, every edit in its history	A saved version (a commit) after each step
The AI call	Its built-in AI, or a key in its secrets	A key in an environment variable
Your data	Its own database, read its rules	A file or a hosted database you pick
Live link	Its publish button	A host, see Prototype to product (M09)

DATA From the two QuackSync builds of 19 and 26 October 2026. Tool menus change, checked 2026-10-26.

WORKSHOP · STEP 3 OF 3

Happy path, then hand it over

3 STEP 3 · 20 MIN

HAPPY PATH, THEN HAND IT OVER

First users today.

→ Log your line, dated, before the first user.
→ Run the happy path: real data in, watch it run, outcome out?
→ Send the link to your Solution space (M05) first buyers. Don't steer.
→ Commit to the bar. Pick floor or full. Set a midweek day.

Output A dated line, users invited, targets, midweek day.

Minutes 20

20:00

ARRIVED EMPTY-HANDED?

Build the smallest slice that works



No winner? Build your best dummy's feature. Back at the problem? Build the smallest working slice, you behind the screen, and invite your interviewees.

HOMEWORK UNTIL NEXT TIME

Put your MVP live, let 10 people per student from your segment use it without your help, and ship 2 fixes for what stopped them.

10 users each

The learning floor is a live MVP per team, 5 such users per student and 1 fix.

Your line: fewer than 3 of your first 10 users reach the outcome by Module 07, and you rebuild the feature. Nobody comes back unprompted, and you bring a pivot.

See you next time!

MODULE 06 · PROTOTYPING WITH AI · CASE DISCUSSION · ABOUT 90 MINUTES

Case discussion: Langfuse

Stand with three founders deciding which borrowed part to replace first as strangers arrive.

01 Previously on Ambitious Entrepreneurship
02 Your presentations
03 **Map it: the parts of every app**
04 **Shrink it: the cheapest version**
05 **Build it: you steer the agent**
06 **Ship it: into users' hands**
07 Wrap-up and check-out
08 Workshop: build it, then hand it over
09 **Case discussion**

VOTE 1 · 16:00 UTC, 29 AUGUST 2023

What do the founders replace or build first?

1. A · Open analytics on an open-source tool for every user
2. B · Integrate OpenTelemetry
3. C · Model-based evaluations and labeling, their stated next step

2:00

TIMELINE · 2023

Fifteen weeks from first commit to strangers

18 May — 15 weeks — 29 August

18 May
Repository created, "kill all", a fresh "init"

26 May
"v1"

1 June
Accounts and a hosted cloud

7 June
Sign-up by form, website button books a call with a founder

26 June
Public sign-up, "Get started by yourself" next to "Guided onboarding"

17 July
Analytics on Looker Studio for flagged users

14 August
Demo project for every new user

20 August
Top product of the day on Product Hunt

29 August, 16:14 UTC
Show HN

DATA Exhibit 1 of the case, from the repositories, the founders' posts and Product Hunt. Langfuse case, Prototyping (M06).

EXHIBIT 3, SELECTED ROWS

What a new user met on 29 August 2023

WHAT	STATE ON 29 AUGUST 2023	SINCE
Start screen	"Create new project" (get started by yourself), "View demo project" and "Guided onboarding" (a call with a founder)	26 June, as on 27 August
Demo project	View-only, live data from the documentation chatbot	14 August
Dashboards	Traces, token usage by model, scores, per-user views, for every user	By 27 August (video)
Analytics (Looker Studio)	For users with the flag "analytics-alpha" only. Others saw "Analytics is in closed alpha" and an invitation to get in touch	17 July
Support	Feedback button, support chat offered while setting up a project, a Discord community	23 June, 29 July
Account	No way to change or reset the password	Open issue of 21 August

DATA Exhibit 3 of the Langfuse case, selected rows, with its source line in the case.

IN THE FOUNDERS' WORDS · 29 AUGUST 2023

Half a day, or a debt?

"Integrating Looker Studio got us to first analytics charts in half a day."

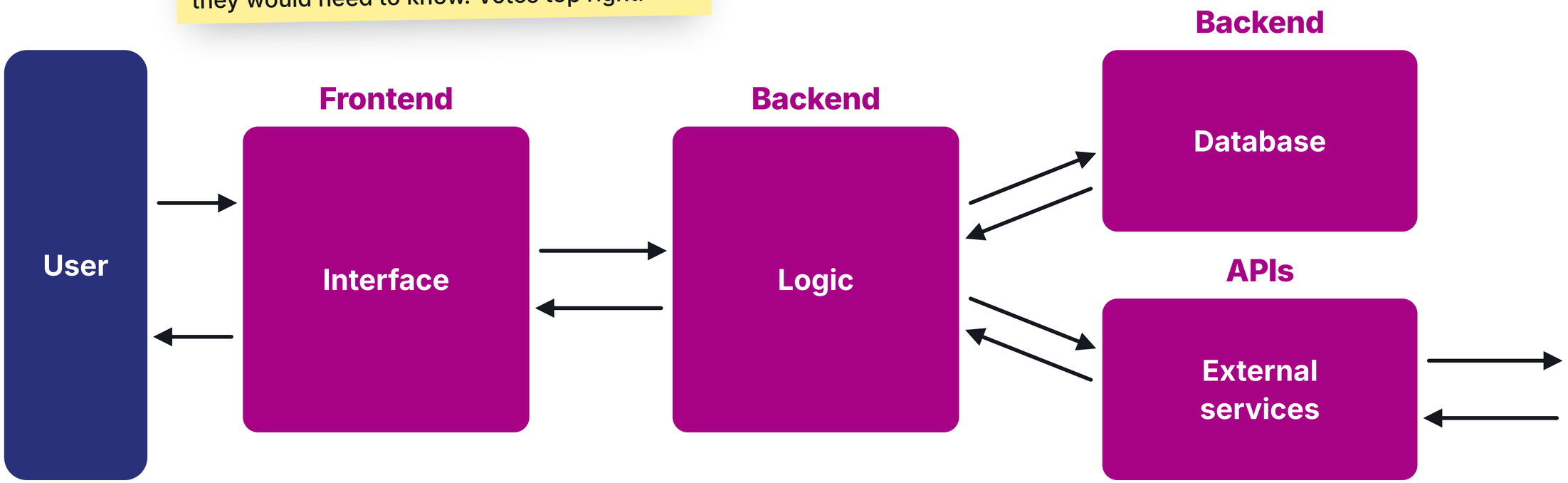
"As it is not open-source and does not work with our UI/UX, we are looking to switch it out for an OSS solution to flexibly generate charts and dashboards."

DATA Marc Klingen, Show HN post, 29 August 2023, as quoted in the case (Q06, Q07). Langfuse case, Prototyping (M06).

BOARD PLAN

Mark what still needs a **founder**

Columns: borrowed and why · what it costs a stranger · what they knew about use · what they would need to know. Votes top right.



USE WITHOUT HELP · ON 29 AUGUST 2023

What did they know about use?

One user, many hours a day
as Deichmann reported it

Five named teams
on the landing page

Alpha users onboarded every week
the July update

370 stars, 21 forks
fifteen minutes after the post

No published number for sign-ups, active users or returning users.

VOTE 2 · SAME QUESTION

What do the founders replace or build first?

1. A · Open analytics on an open-source tool for every user
2. B · Integrate OpenTelemetry
3. C · Model-based evaluations and labeling, their stated next step

2:00

BRIDGE · TO YOUR OWN BUILD

Mark what still needs you

Langfuse let strangers in with a call, a chat and a flag still in place. Now draw your own MVP on the four parts and mark what still needs you.

See you next time!